

ISO/IEC 15504 is intended to harmonize the many different approaches to software process assessment. It has nine parts:

- Part 1 : *Concepts and introductory guide*
- Part 2 : *A reference model for processes and process capability*
- Part 3 : *Performing an assessment*
- Part 4 : *Guide to performing assessments*
- Part 5 : *An assessment model and indicator guidance*
- Part 6 : *Guide to competency of assessors*
- Part 7 : *Guide for use in process improvement*
- Part 8 : *Guide for use in determining supplier process capability*
- Part 9 : *Vocabulary*

The reference model in Part 2 documents the set of universal software engineering processes that are fundamental to good software engineering and that cover best practice activities. It describes processes that an organization may perform to acquire, supply, develop, operate, evolve and support software and the process attributes that characterize the capability of those processes. The purpose of the reference model is to provide a common basis for different models and methods for software process assessment, ensuring that results of assessments can be reported in a common context. The reference model architecture is two-dimensional. The *process dimension* is characterized by process purpose statements, which are the essential measurable objectives of a process. The processes are listed in Table 4.

Table 4: Mapping Between ISO/IEC 15504-2 Processes and Software CMM Key Process Areas

(Note: For full description of each of the Activities of Software CMM v1.1, readers should refer to Ref [2].)

ISO/IEC 15504 Processes	Software CMM v1.1
CUS.1 Acquisition Software Subcontract Management	Software Subcontract Management
CUS.1.1 Acquisition preparation	Software Subcontract Management, Activity 1
CUS.1.2 Supplier selection	Software Subcontract Management, Activity 2
CUS.1.3 Supplier monitoring	Software Subcontract Management, Activities 5 and 7-11
CUS.1.4 Customer acceptance	Software Subcontract Management, Activity 12
CUS.2 Supply	(Software Project Planning; Software Project Tracking & Oversight; Software Product Engineering)
CUS.3 Requirements elicitation	---
CUS.4 Operation	---
CUS.4.1 Operational use	---
CUS.4.2 Customer support	---
ENG.1 Development	Software Product Engineering

ISO/IEC 15504 Processes Software CMM v1.1

Software CMM v1.1

ENG.1.1	System Requirements Analysis and Design	-----
ENG.1.2	Software requirements	Software Product Engineering, Activity 2
ENG.1.3	Software design	Software Product Engineering, Activity 3
ENG.1.4	Software construction	Software Product Engineering, Activity 4
ENG.1.5	Software integration	Software Product Engineering, Activity 6
ENG.1.6	Software testing	Software Product Engineering, Activity 7
ENG.1.7	System integration and Testing	(Software Product Engineering, Activities 6 and 7)
ENG.2	System and software Maintenance	-----
SUP.1	Documentation	Software Product Engineering, Activity 8
SUP.2	Configuration management	Software Configuration Management
SUP.3	Quality assurance	Software Quality Assurance
SUP.4	Verification	(Peer Reviews; Software Product Engineering, Activities 5 and 6)
SUP.5	Validation	Software Product Engineering, Activity 5
SUP.6	Joint review	Software Project Tracking & Oversight, Activity 13
SUP.7	Audit	(Software Quality Assurance)
SUP.8	Problem resolution	Software Configuration Management, Activity 5
MAN.1	Management	(Software Project Planning; Software Project Tracking & Oversight; Integrated Software Management)
MAN.2	Project management	Software Project Planning; Software Project Tracking & Oversight; Integrated Software Management
MAN.3	Quality management	Software Quality Management
MAN.4	Risk management	Software Project Planning, Activity 13; Software Project Tracking & Oversight, Activity 10; Integrated Software Management, Activity 10
ORG.1	Organizational alignment	-----
ORG.4	Infrastructure	Organization Process Definition
ORG.2	Improvement	Organization Process Definition
ORG.2.1	Process establishment	Organization Process Definition
ORG.2.2	Process assessment	Organization Process Focus, Activity 1
ORG.2.3	Process improvement	Organization Process Focus; (Process Change Management)
ORG.3	Human resource management	Training Program
ORG.4	Infrastructure	-----
ORG.5	Measurement	Measurement and Analysis (common feature)
ORG.6	Reuse	Requirements Management Inter-group Coordination Peer Reviews Quantitative Process Management Defect Prevention Technology Change Management Process Change Management

The mapping in Table 4 shows how the topics in ISO/IEC 15504 relate to the equivalent topics/*key process areas* in the Software CMM. Topics are typically not isomorphic but are highly correlated. Anyone adequately implementing, for example, the Configuration Management Process in ISO/IEC 15504 could reasonably expect to have satisfied the Software Configuration Management key process area in the Software CMM. Topics are not usually isomorphic because of extensions that may have been added or different levels of abstraction that may have been chosen (e.g., the Development Process in ISO/IEC).

9.5 Types of Capability Maturity Models (CMMs)

The Software Capability Maturity Model is the most famous one. However, the fact that there are a number of other Capability Maturity Models not known to many people. In this section, we have compiled such information on other CMM products (Table 5). The source of this data is <ftp://ftp.sei.cmu.edu/public/documents/> or <http://www.sei.cmu.edu/>

Table 5: Types of Capability Maturity Models (CMMs)

Type of CMM	Description	Documents
Software CMM (the original CMM)	Key Practices of the Capability Maturity Model	CMU/SEI-93-TR-025
	Capability Maturity Model for Software (SW-CMM)	CMU/SEI-93-TR-024
	Personal Software Process (PSP)	Published books
Software Acquisition CMM	Software Acquisition Capability Maturity Model (SA-CMM)	CMU/SEI-99-TR-002
	Software Acquisition Improvement Framework (SAIF) Definition	CMU/SEI-98-TR-003
CMM-Based Assessment	CMM Appraisal Framework (CAF)	CMU/SEI-95-TR-001
	Software Capability Evaluation (SCE) Method Description	CMU/SEI-96-TR-002
	CMM -Based Appraisal for Internal Process Improvement (CBA IPI): Method Description	CMU/SEI-96-TR-007
CMM Framework	Software Process Framework for the SEI Capability Maturity Model	CMU/SEI-94-HB-001
	Software Process Framework for the SEI Capability Maturity Model: Repeatable Level	CMU/SEI-93-SR-007
	How to Use the Software Process Framework	CMU/SEI-97-SR-009
	Common CMM Framework	—

Type of CMM	Description	Documents
Personnel Management	People Capability Maturity Model (P-CMM)	CMU/SEI-95-MM-002
	Overview of the People Capability Maturity Model, Version 1.1	CMU/SEI-95-MM-001
	People CMM-Based Assessment Method Description	CMU/SEI-98-TR-012
Systems Engineering	Systems Engineering Capability Maturity Model (SE-CMM)	CMU/SEI-95-MM-003
	Description of the Systems Engineering Capability Maturity Model Appraisal Method	CMU/SEI-96-HB-004
	Integrated Product Development CMM (IPD-CMM)	None—product cancelled
Integrated CMM	SEI Integrated CMM (CMMI)	Not public
	FAA Integrated CMM (FAA-iCMM) Federal Aviation Administration Integrated Capability Maturity Model (FAA-iCMM) Appraisal Method Available from http://www.faa.gov/ait/ait5/FAA-iCMM.htm	
Security	Security Systems Engineering CMM (SSE-CMM)	Not public
	Trusted CMM (T-CMM) (Replaces some KPA's from the SW-CMM; not a stand-alone CMM)	Not public
Related International Standards (available from IEEE Standards site)	ISO/IEC DTR 15504—Software Process Assessment	---
	ISO/IEC 12207—Software life cycle processes	---

9.6 The CMM-Integrated Model (CMM-I)

9.6.1 What is CMMI?

In this section, we'll discuss the overview and model structure of the Integrated Capability Maturity Model, famously known as the CMMi. Then we'll discuss the two representations available for this model. Next we'll provide a discussion on comparative differences between the Software CMM model vis-à-vis the Integrated CMM model.

CMMI (Capability Maturity Model—Integrated) is the newest process capability model to be provided by the Software Engineering Institute. It is a model, which integrates and replaces multiple CMMs (we discussed some of this in section 9.5) into a single model in order to reduce duplication and eliminate redundancy. Common terminology and components provide a consistent style. It provides guidance on Systems and Software Engineering and encourages process improvements in organizations of any structure.

The SEI, government and multiple industries and organizations built the CMMI. It was sponsored by the Department of Defense as the CMMI Integration Project. The CMMI incorporates models for systems engineering, software engineering, software acquisition, workforce practices and integrated product and process development.

9.6.2 The Background to the CMM-I Model

The CMMI Integration Project was formed to sort out the problem of using multiple CMMs. The initial mission of the project was to combine three source models into a single model for use by organizations pursuing enterprise-wide process improvement.

- CMM for Software (SW-CMM V2.0 draft C)
- Electronic Industries Alliance/Interim Standard (EIA/IS) 731 (the Systems Engineering standard)
- Integrated Product and Process Development Capability Maturity Model (IPD-CMM) V 0.98

The following diagram (Figure 2) shows the evolution of multiple process models into the CMMI model. As seen in the diagram, the resulting framework accommodates multiple disciplines and is flexible enough to support two different representations:

- Staged
- Continuous

It supports future integration of other discipline-specific CMMI models and is consistent and compatible with the ISO/IEC 15504 technical report for software process assessments. The integrated model can be adopted by those currently using other CMMs as well as those new to the CMMI concept. Figure 3 depicts the components of the Integrated CMM Model (*Ref. CMMI SE/SW/IPPD, Version 1.1 page 10*)

9.6.3 Types of CMM-I Models

The URL to the CMMi official website by the SEI (Software Engineering Institute) is www.sei.cmu.edu/cmmi/ and we urge the readers to visit this site. A number of latest products and services by the SEI are listed here. This site mention the four models of the Integrated CMM, each with two kinds of representation:

- CMMI-SE/SW/IPPD/SS, V1.1 (Continuous Representation)
- CMMI-SE/SW/IPPD/SS, V1.1 (Staged Representation)

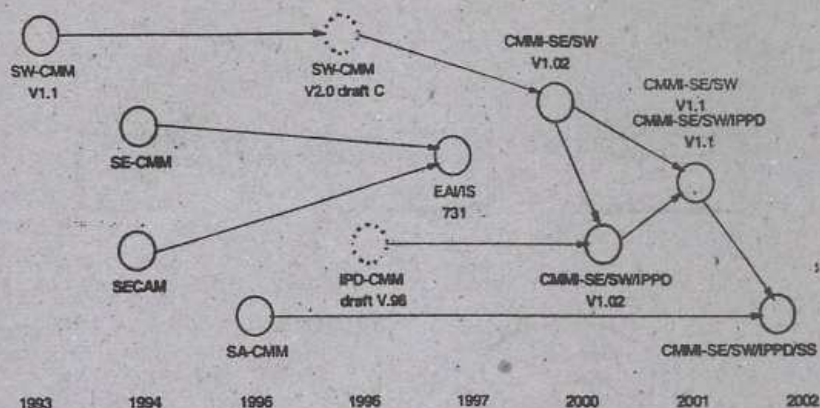


Figure 2: Evolution of CMM-I From Source Models

Source: CMMIsm—The Evolution Continues (SEI Abstract September 2001)
 Pamela Curtis, David Michael Phillips, and Joan Weszka

- CMMI-SE-SW-IPPD V1.1 (Continuous Representation)
- CMMI-SE-SW-IPPD V1.1 (Staged Representation)
- CMMI-SE-SW V1.1 (Continuous Representation)
- CMMI-SE-SW V1.1 (Staged Representation)
- CMMI-SW V1.1 Continuous Representation)
- CMMI-SW V1.1 (Staged Representation)

Readers will note that each CMMI model is given a name consisting of "CMMI-" followed by the abbreviation for the disciplines selected for that model. Where more than one discipline is modeled, the disciplines are listed with a slash (/) between them. This is as follows:

- CMMI-SE/SW is the name given to the *systems engineering and software engineering integrated model*.
- CMMI-SE/SW/IPPD is the name given to the *systems engineering, software engineering, and integrated product and process development integrated model*.
- CMMI-SE/SW/IPPD/SS is the name given to the *systems engineering, software engineering, integrated product and process development, and supplier sourcing integrated model*.

Which model to choose and with which representation can be a very involved decision requiring pondering by all the relevant stakeholders. At times an organization may need involvement of an external consultant to help make the decision. One such article that discusses some of these aspects is "CMMI Myths and Realities" by Lauren Heinz available at ref [10]. In that article he has discussed the often-made

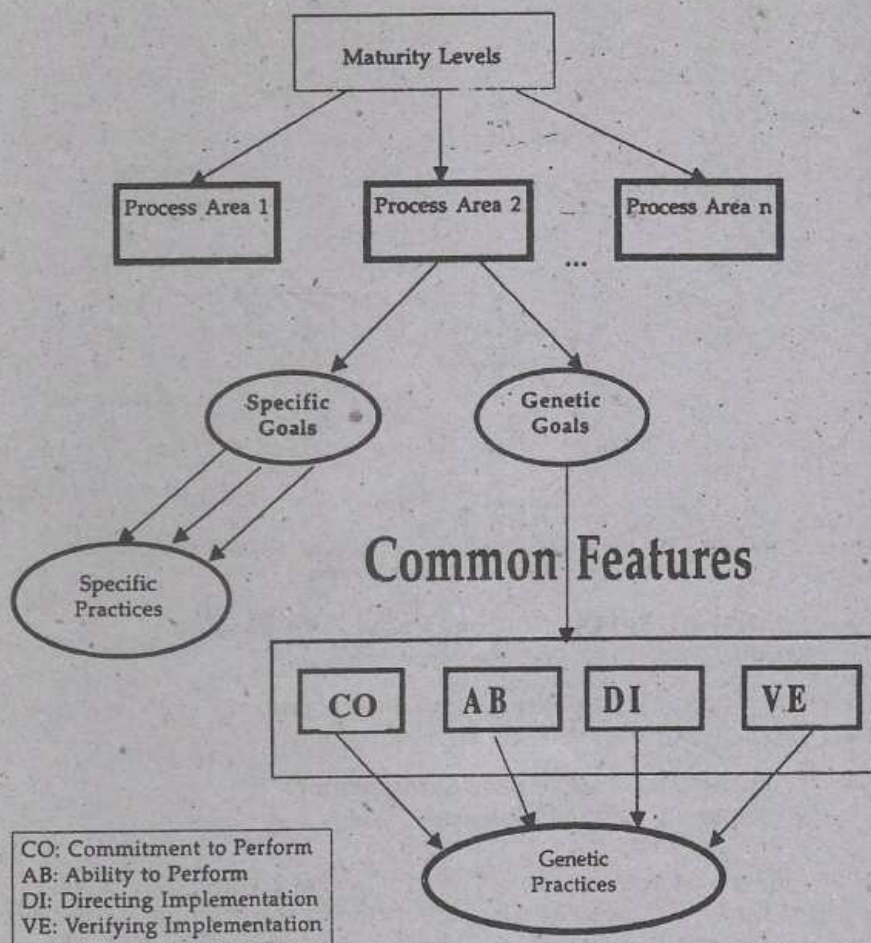


Figure 3: CMM-Integrated Model Components

remarks: CMMI is too big and complex, A CMMI appraisal takes longer and costs more and CMMI is for large organizations. In the next section, we have provided a brief discussion on "Choosing a Representation" for a CMM-I Model.

9.6.4 Transitioning to CMM-I Model

A large number of organizations worldwide are already assessed to be at the various levels of SW CMM. Some organizations may be making the decision to transition to the CMM-Integrated model as the current SW CMM model is to sunset soon. (For exact details of this, readers can visit site www.sei.cmu.edu. We recommend readers to have a look at the CMMI White Paper quoted under reference [12])

In general, while making the transition decision, a number of factors need to be considered; they are listed below:

- Size of organization in terms of total number of employees
- Type of organization (in terms of software business conducted)
- Whether the organization is into pure product development activities or service providing
- The complexity of organizations products and services
- The Process Improvement Models already adopted by the organization (if any)
- Organization's familiarity with the CMM-Integrated Model
- What information the organization has used to aid the decision of transitioning to CMM-Integrated Model
- The type of CMM-I model representation the organization intends to use and the reasons behind
- People, Technology and Process related support needed by the organization to transition to the Integrated CMM model

This is why we said earlier that the decision to make a transition from SW CMM to CMM Integrated model depends on a number of factors and being a decision with long term implications, involves all the relevant stakeholders and at times, also the services of an external consultant.

9.6.5 Process Maturity Levels in CMM-I Model

The staged representation of the CMM-I is the same representation used for the Software CMM. It provides 5 levels of process maturity:

1. Initial
2. Managed
3. Defined
4. Quantitatively Managed
5. Optimizing

These are shown in Figure 4.

Each *maturity level* contains a predefined set of *Process Areas*. In order to achieve a specific maturity level, all *goals* within the set of process areas for that maturity level must be attained. The *staged representation* provides a predefined road map for process improvement, each level building on the maturity of the previous level. The staged representation allows comparison of process maturity levels across organizations. Figure 5 (Chart-A) shows the grouping of Process Areas into the 5 levels of the *Staged Representation*.

The continuous representation is the same representation used for the systems engineering model (EIA/IS 731) and the organization of the Process Areas is derived from ISO/IEC 15504. It groups the process areas by categories of common disciplines:

CMMI Maturity Levels

- 5 Process improvement is a continuous focus.
- 4 Processes are both measured and controlled
- 3 Processes are characterized for the organization and is proactive
- 2 Processes are characterized for projects and actions are often reactive.
- 1 Processes are unpredictable, poorly controlled and reactive.

Defined

Managed

Optimizing

Quantitatively
Managed

Initial

- Process Management processes
- Project Management processes
- Engineering processes
- Support processes

Figure 5 (Chart B) shows the grouping of Process Areas into the Continuous Representation process categories.

9.6.6 Choosing a CMMI Model Representation

As mentioned earlier, there are two "representations" of each CMMI discipline: *staged* and *continuous*. A representation reflects the organization, use, and presentation of model elements. Both representations contain essentially the same information. Each representation consists of process areas that contain a purpose statement, introductory text, *specific goals*, *specific practices*, *generic goals*, and *generic practices*. For more information about these components, refer to Figure 3, "Model Components" of the CMMI model.

Staged—The *staged representation* offers a roadmap to approach process improvement: one predetermined step at a time. *Process areas* (The PAs) are grouped at *maturity levels* that provide organizations with a proven approach for process improvement. The staged representation prescribes the order of implementation for each process area according to maturity levels. Achieving each maturity level ensures that an adequate

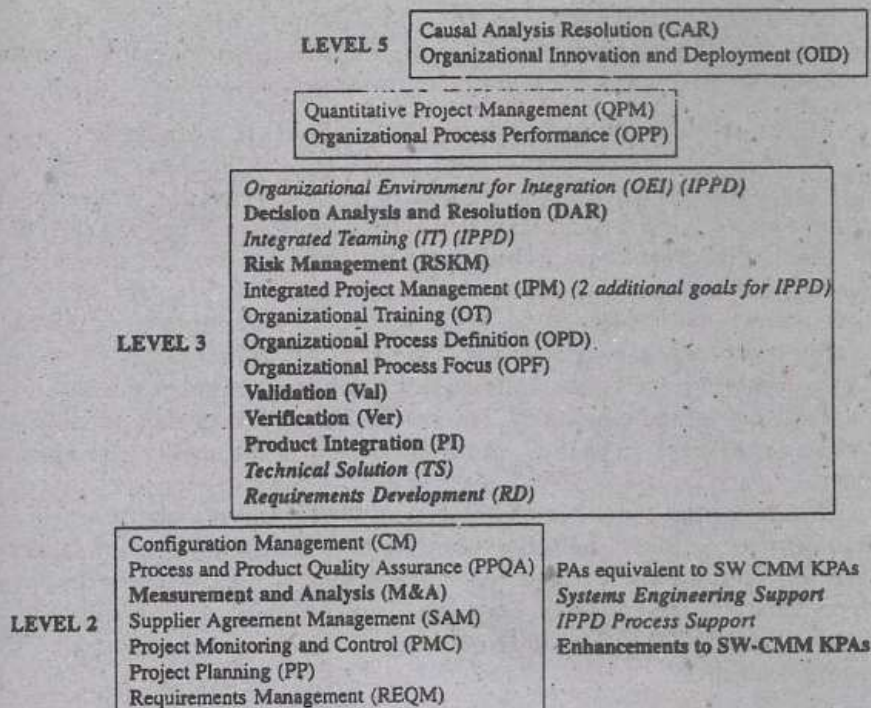


Chart A: Staged Representation

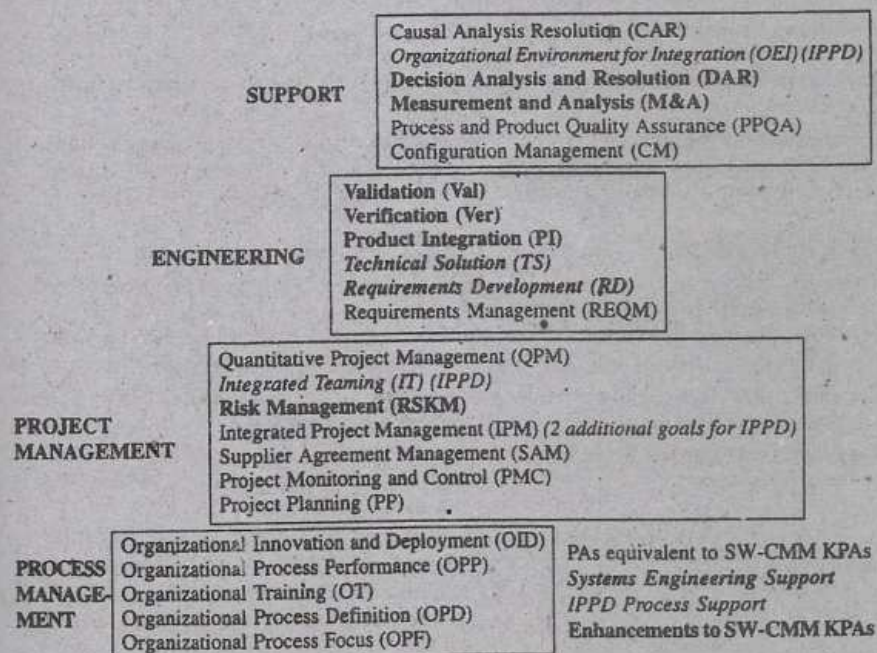


Chart B: Continuous Representation

Figure 5: Grouping of Process Areas

improvement foundation has been laid for the next maturity level, thus minimizing the organization's process improvement investment and risk while maximizing the benefits to the organization.

Continuous—The *continuous representation* offers a more flexible approach to *process improvement*. It is designed for organizations that would like to choose a particular *process area* or set of process areas based on trouble spots in the organization or a set of process areas that are closely aligned to the organization's business objectives. Process-improvement objectives are mapped to process areas in the model to identify the process areas to be implemented. As a process area is implemented, the *specific practices* and *generic practices* are grouped into capability levels. These *capability levels* enable the organization to implement the chosen process area(s) incrementally. The continuous representation also allows an organization to implement different process areas at different rates.

Readers might wonder as to what are the differences between the two representations. The table 6 outlines the differences between the two representations. Various factors affect the decision to choose an appropriate representation. They are discussed next. First we should carefully study Table 6 to understand the differences between the two representations.

Table 6: Differences between Staged and Continuous Representations of CMM-I

Continuous Representation	Staged Representation
Process areas are organized by process area categories.	Process areas are organized by maturity levels.
Improvement is measured using capability levels that reflect incremental implementation of a particular process area.	Improvement is measured using maturity levels that reflect the concurrent implementation of multiple process areas.
There are six capability levels, 0-5.	There are five maturity levels, 1-5.
There are an N+ number of practices because there are two types of specific practices: base and advanced.	There are an N number of practices because there is only one type of specific practice. The concept of advanced practices is not used, but is addressed through other means.
Capability levels are used to organize the generic practices.	Common features are used to organize the generic practices.
All generic practices are listed in each of the process areas.	Only the generic practices that are applicable to that maturity level are listed in the process areas at that level.
Generic practices exist for capability levels 1-5.	Generic practices exist for maturity levels 2-5. A subset of generic practices used in the continuous representation are applied to each process area based on its maturity level.
Overview text is written to describe the continuous representation.	Overview text is written to describe the staged representation.

An additional appendix describing equivalent staging is included, which allows a translation of a target profile into a maturity level.

There is no equivalence concept that allows a translation of maturity levels into a target profile.

Choosing the representation certainly requires some thought; in fact many find this rather perplexing and so a brief discussion on this should be in order. According to the Software Engineering Institute (SEI), three categories of factors may influence your decision- business, cultural, and legacy:

Business Factors: An organization with mature knowledge of its business objectives is likely to have a strong mapping of its processes to its business objectives. An organization such as this may find the continuous representation more useful to assess its processes and to determine how well the organization's processes support and meet business goals. The staged representation is widely used and maturity level ratings are often published. If your organization is concerned about benchmarking with your competitors and/or publishing results, the staged representation might be selected.

Cultural Factors: Cultural factors to consider when selecting a representation have to do with an organization's ability to deploy a process improvement program. For instance, an organization might select the continuous representation if the culture is experienced in process improvement or has a specific process that needs to be improved quickly. An organization that has little experience in process improvement might choose the staged representation, which provides additional guidance about the order in which changes should occur.

Legacy: Organizations with a strong systems engineering culture might be more familiar with the continuous representation, whereas software organizations may be more accustomed to a staged representation. If an organization has experience with a staged representation, it may be wise to continue with the staged representation of CMMI, especially if it has invested resources and deployed processes across the organization that are associated with a staged representation. The same is true for an organization that has experience with a continuous representation. Both staged and continuous representations were made available so that the communities that have used the different representations successfully could continue in a manner that is comfortable and familiar to them.

It should be noted that an organization isn't forced to select one representation over another. In fact, an organization may find utility in both representations. It is rare that an organization will implement either representation exactly as prescribed.

9.7.7 Comparison: SW CMM v/s CMM-Integrated

"Key Process Areas (KPAs)" will now be called "Process Areas (PAs)". The software CMM has 18 KPAs. CMMI has 24 PAs. The software CMM KPAs have goals and key practices which support those goals. Key practices are composed of Common Features and Activities Performed (AC). Common Features consist of Commitment to Perform (CO), Ability to Perform (AB), Measurements and Analysis (ME), and Verifying Implementation (VE).

There are many similarities across the common features in the software CMM KPAs, however, they are not exactly the same for all KPAs. In the CMMI model, each process area has goals, but they are comprised of General Goals and Specific Goals. General practices support the General Goals, which consist of Commitment to Perform (CO), Ability to Perform (AB), Directing Implementation (DI), and Verifying Implementation (VE). Directing Implementation replaces the Measurements and Evaluation key practice of the software CMM model. Generic Goals apply to all process areas, so there is one consistent set of practices across all process areas. This is equivalent to and an improvement to the "similar" Common Features in the software CMM. Specific Goals consist of Specific Practices, which are equivalent to the Activities Performed in the software CMM.

A frequently asked question is "How do the software CMM KPAs relate to the CMMI PAs?" The chart showing the mapping is given in Figure 6.

9.7 Other Models for Software Process Improvement and Performance Excellence

Having done extensive discussion on the SW CMM model and the Integrated CMM model, we are now going to list down the other "software improvement models" that are available. We have already mentioned the various types of "Capability Maturity Models" (CMMs) available. We explained the ISO 9001 standard as compared to the SW CMM model. We also discussed the ISO 15504 in relation to the CMM. The ISO standard will be discussed in a separate chapter.

9.7.1 TickIT

TickIT is about improving the quality of software and its application. TickIT is supported by the UK and Swedish software industries. An important purpose of TickIT has been to simulate software system developers to think about:

- What quality really is in the context of the processes of software development,

A comparison shows differences and similarities...SW-CMM (18 KPAs)Maturity Level 2

- Requirements Management
- Software Project Planning
- Software Project Tracking & Oversight
- Software Subcontract Management
- Software Quality Assurance
- Software Configuration Management

Maturity Level 3

- Organization Process Focus
- Organization Process Definition
- Training Program
- Software Product Engineering

- Peer Reviews
- Integrated Software Management
- Intergroup Coordination

Maturity Level 4

- Quantitative Process Management
- Software Quality Management

Maturity Level 5

- Defect Prevention
- Technology Change Management
- Process Change Management

CMMI (24 PAs)Maturity Level 2

- Requirements Management
- Project Planning
- Project Monitoring and Control
- Supplier Agreement Management
- Process and Product Quality Assurance
- Configuration Management
- Measurement and Analysis

Maturity Level 3

- Organizational Process Focus
- Organizational Process Definition
- Organizational Training
- Requirements Development
- Technical Solution
- Product Integration
- Verification
- Validation
- Integrated Project Management (IPPD)
- Risk Management
- Integrated Teaming
- Organization Environment for Integration
- Decision Analysis and Resolution

Maturity Level 4

- Quantitative Project Management
- Organizational Process Performance

Maturity Level 5

- Causal analysis and Resolution
- Organizational Innovation and Deployment

Process Areas equivalent to SW-CMM KPAs, *Systems Engineering Support*, *IPPD Support*, *Enhancements to SW-CMM KPAs*

Figure 6: SW CMM and CMM-I: Comparison

- How quality may be achieved, and
- How quality management systems may be continuously improved

Although certification of compliance to ISO 9001 is a contractual requirement for software suppliers in certain market areas, it should be a by-product of the more fundamental aims of quality achievement and improvement, and the delivery of customer satisfaction. With regard to the certification itself, the objectives of TickIT are to:

- Improve market confidence in third party quality management system certification through accredited certification bodies for the software sector,
- Improve professional practices amongst quality management system auditors in the software sector,
- Publish authoritative guidance material (the TickIT Guide) for all stakeholders

TickIT applies to all types of information systems supply which involve software development processes. Typical systems suppliers include system houses, software houses and in-house developers. TickIT disciplines are also relevant for the development of embedded software. In the UK, TickIT is recognized by all Government departments and major purchasers and it is compatible with European requirements for accredited quality system certification. According to a statistics, world-wide, more than 1400 TickIT certifications have been issued so far. TickIT is managed and maintained by DISC, the department within BSI (British Standards Institute) with responsibility for all aspects of standardization in information systems and communications.

9.7.2 ISO 12207

ISO 12207 offers a framework for software life-cycle processes from concept through retirement. It is especially suitable for acquisitions because it recognizes the distinct roles of acquirer and supplier. In fact, the standard is intended for two-party use where an agreement or contract defines the development, maintenance, or operation of a software system. It should be noted that ISO 12207 is *not applicable* to the purchases of "commercial-off-the-shelf (COTS) software products.

ISO 12207 provides a structure of processes using mutually accepted terminology, rather than dictating a particular life-cycle model or software development method. It describes five "**primary processes**"—ACQUISITION, SUPPLY, DEVELOPMENT, MAINTENANCE and OPERATION. It divides these five processes into "activities", and the activities into "tasks", while placing requirements upon their execution. ISO 12207 also specifies eight "**supporting processes**"—DOCUMENTATION, CONFIGURATION MANAGEMENT, QUALITY ASSURANCE, VERIFICATION, VALIDATION, JOINT REVIEW, AUDIT, and PROBLEM RESOLUTION. It also describes four "**organizational management processes**"—MANAGEMENT, INFRASTRUCTURE, IMPROVEMENT, and TRAINING.

Since it is a relatively high-level document, 12207 does not specify the details of how to perform the activities and tasks comprising the processes. Nor does it prescribe the name, format or content of documentation. It expects organizations to tailor the seventeen processes mentioned above (5 primary processes, 8 supporting processes and 4 supporting processes.) to fit the scope of their particular projects by deleting all inapplicable activities. It defines 12207 compliance as the performances of those processes. Considering this, organizations seeking to apply 12207 may want to use additional standards or procedures that specify those details.

9.7.3 IEEE 1074

The name of this process improvement framework is "IEEE Standard for Developing Software Life Cycle Processes. The standard describes the responsibilities associated for developing a software life cycle:

- Available life-cycle models (prototyping, spiral, waterfall etc.)
- One life-cycle model is selected for a project, or set of projects
- The activities described in IEEE 1074 are mapped, in time order; onto the life-cycle model. Owners and a schedule are assigned to each activity. The activities described in IEEE 1074 are a minimum set of mandatory activities.

This standard is primarily written for a process architect, the individual responsible for establishing the software life cycle to be followed on projects. By now IEEE must have revised this standard to harmonize it with ISO 12207.

9.7.4 Malcolm Baldrige National Quality Award (MBNQA)

The Malcolm Baldrige National Quality Award (MBNQA) is an annual award to recognize United States companies for performance excellence (not necessary restricted to software processes related excellence).

The MBNQA is a "Total Quality Management" (TQM) often referred to as a framework. The model is made of 7 elements (shown below in *italic*) under four basic principles (shown below in **bold**):

- i. **Driver:** The 1.0 *Leadership* element of MBNQA drives the system by
 - Setting directions
 - Creating values, goals and systems
 - Guiding the pursuit of customer value and company performance improvement
- ii. **System:** Composed of processes for meeting the company's customer and performance requirements – elements, include:
 - 2.0 *Information and Analysis*
 - 3.0 *Strategic Planning*
 - 4.0 *Human Resource Development and Management*
 - 5.0 *Process Management*
- iii. **Measures of Progress:** These measures provide a result-oriented basis for channeling actions to delivering ever-improving customer value and company performance. These measure are addressed in element 6.0 *Business Results*, and include:
 - Product and Service quality
 - Productivity improvement
 - Waste Reduction/Elimination
 - Supplier Performance
 - Financial Results

- iv. **Goal:** The basic aims of the system driver to deliver the ever-improving customer values and company performance. These goals are addressed in *7.0 Customer Focus and Satisfaction*, and include:
- Customer satisfaction
 - Customer satisfaction relative to competitors
 - Customer retention
 - Market Share Gain

Other countries award equivalent to the MBNQA are:

- The Canada Award of Excellence (discussed briefly in the next section)
- The Business Excellence Award of Europe

Comparing MBNQA against ISO 9001 and SEI SW-CMM v1.0

Each standard and model emphasizes different aspects of TQM (Total Quality Management). All the three (MBNQA, ISO and SW CMM) standards and models have an overlapping focus on *process management*. A study was done by Michael Tingey to focus on the types of activities emphasized by each of the three models mentioned above. The study showed that the MBNQA model puts a large emphasis on *improving* and then *planning*. In comparison, the ISO 9001 standard puts its highest emphasis on *managing* and then on *implementing*. The SEI SW-CMM model is the reverse of ISO 9001—its largest emphasis is on *implementing* then *managing*.

9.7.5 Canada Award for Excellence

We had mentioned about this award in the section on MNBQA. The Canada Award for Excellence (CAE) is an annual award to recognize Canadian companies for performance excellence. It is a Total Quality Management (TQM) model. It is made up of 7 elements (shown below in *italic*).

1.0 Leadership element of CAE focuses on those who have primary responsibility and accountability for the organization's performance; this element's primary focus is on the organization's

- Strategic direction
- Leadership involvement
- Outcomes
- Continuous improvement

2.0 Planning element of CAE pertains to the planning process in regards to improvement, the linkage of planning to strategic direction/intent, and the measurement of performance to assess progress; this element's primary focus is on the organization's

- Development and content
- Assessment
- Outcomes
- Continuous improvement

3.0 Customer Focus element of CAE pertains to the organization's focus on customer driven innovation and on the achievement of customer satisfaction; this element's primary focus is on the organization's

- Voice of the customer
- Management of customer relationships
- Measurement of customer satisfaction
- Outcomes
- Continuous improvement

4.0 People Focus element of CAE pertains to the development and deployment of a human resource plan for meeting the goals of the organization, and achievement excellence through resource plan for meeting the goals of the organization, and achieving excellence through people. In addition, the element pertains to the organization's efforts to foster and support an environment that encourages people to teach their full potential; this element's primary focus is on the organization's

- Human resource planning
- Participatory environment
- Continuous learning
- Employee satisfaction
- Outcomes
- Continuous improvement

5.0 Process Management element of CAE pertains to how work is organized to support the organization's strategic direction, with a specific focus on quality assurance practices, as well as continuous improvement; this element's primary focus is on organization's

- Process definition
- Process control
- Process improvement
- Outcomes
- Continuous improvement

6.0 Supplier Focus element of CAE pertains to the organization's external relationships with other organizations, institutions and/or alliances that are critical to it meeting its strategic objectives; this element's primary focus is on the organization's

- Partnering
- Outcomes
- Continuous improvement

7.0 *Organizational Performance* element of CAE pertains to the outcomes from the overall efforts for improvement, and their impact on organizational achievement; this element's primary focus is on the organization's

- Service/product quality
- Operational results
- Customers and marketplace
- Employee satisfaction and morale
- Financial performance

9.7.6 The EFQM Excellence Model

The EFQM Model is a "non-prescriptive framework" that recognizes there are many approaches to achieving "sustainable excellence". The Model has 9 criteria depicted in Figure 7. Five of these criteria are called "Enablers" and four are "Results". The "Enabler" criteria cover what an organization does. The "Results" criteria cover what an organization achieves. "Results" are caused by "Enablers". The model is based on the premise that: **Excellent results with respect to Performances, Results, Customers, People and Society are achieved through Partnerships & Resources, and Processes.**

The 9 boxes of the EFQM Excellence Model, depicted in Figure 7, represent the criteria against which to assess an organization's progress towards excellence. Each of the nine criteria has a definition, which

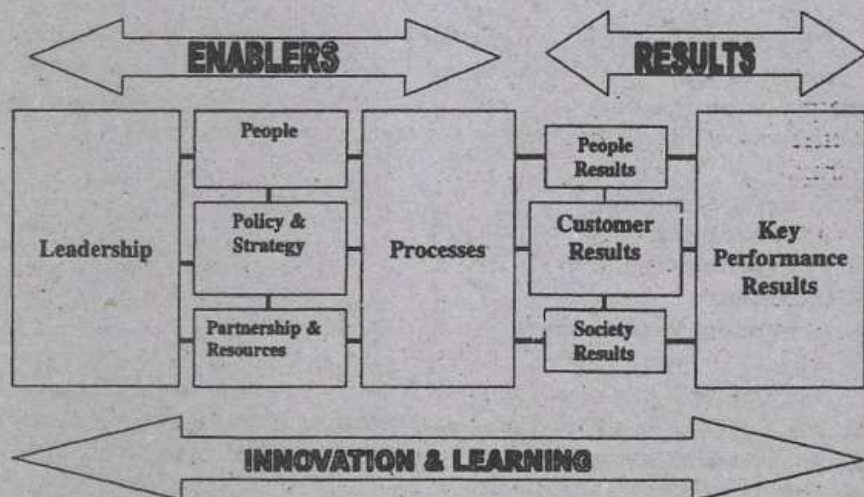


Figure 7: The EFQM Excellence Model

explains the high level of meaning of that criterion. To develop the high level meaning further, each criterion is supported by a number of sub-criteria. Sub-criteria pose a number of questions that should be considered in the course of an assessment. Finally, below each sub-criterion are list of possible areas to address. The areas to address are not mandatory nor are they exhaustive lists but are intended to further exemplify the meaning of the sub-criterion. (For additional details on this, users can refer to model.info@efqm.org)

Within this non-prescriptive framework approach there are some "Fundamental Concepts", which underpin the EFQM Model. These are expressed below. There is no significance intended in the order in which the concepts are presented below. The list (Fundamental Concepts of EFQM Excellence Model) is not meant to be exhaustive and they will change as excellent organizations develop and improve. Fundamental Concepts of EFQM Excellence Model are described below:

Results Orientation—Excellence is dependent upon balancing and satisfying the needs of all relevant stakeholders (this includes the people employed, customers, suppliers and society in general as well as those with financial interests in the organization).

Customer Focus—The customer is the final arbiter of product and service quality. Customer loyalty, retention and market share gain are best optimized through a clear focus on the needs of the current and potential customers.

Leadership & Constancy of Purpose—The behaviour of an organization's leaders creates a clarity and unity of purpose within the organization and an environment in which the organization and its people can excel.

Management by Processes and Facts—Organizations perform more effectively when all inter-related activities are understood and systematically managed and decisions concerning current operations and planned. Improvements are made using reliable information that includes stakeholder perceptions.

People Development & Involvement—The full potential of an organization's people is best released through shared values and a culture of trust and improvement, which encourages the involvement of everyone.

Continuous Learning, Innovation & Improvement—Organizational performance is maximized when it is based on the management and sharing of knowledge within a culture of continuous learning, innovation and improvement.

Partnership Development—An organization works more effectively when it has mutually beneficial relationships, built on trust, sharing of knowledge and integration, with its Partners.

Public Responsibility—The long-term interest of the organization and its people are best served by adopting an ethical approach and exceeding the expectations and regulations of the community of large.

9.7.7 Other Not-so-Well Known Models for Process Improvement

So far, we discussed some of the most popular and established approaches—CMM, ISO 9000 and ISO/IEC 15504 etc. In this section, we provide an overview of other models, which may not be so known, to most readers. This is not an exhaustive list but gives some idea to the reader as to what the other process improvement models are. To start with, there is *Engineering Maturity Model (EMM)*, *Software Development Capability/Capacity Review (SDC/CR)*, *Systems Security Engineering (SSE)*, *Software Technology Diagnostic (STD)*, *Self-Assessment Method (SAM)*, *Quantum*. Apart from these, there are some more—brief information on those is provided in the table 7.

9.8 The People Maturity Model (P-CMM)

The People Maturity Model (P-CMM) is of a different genre altogether. Whereas the focus of Software CMM and the Integrated CMM is on the maturity of software related practices, the focus of P-CMM is on the maturity of *the people related practices* (in the software organizations). Details of this model are available at the SEI site www.sei.cmu.edu. Here, we present only an overview for the sake of completeness in the discussion of process improvement models discussed in this chapter.

9.8.1 Need for the People Capability Maturity Model

In order to improve their performance, organizations must focus on three interrelated components—*people, process, and technology*. With the help of the Capability Maturity Model for Software (discussed in section 9.2), many software organizations have made cost-effective, lasting improvements in their software processes and practices. Yet many of these organizations have discovered that their continued improvement requires significant changes in the way they manage, develop, and use their people for developing and maintaining software and information systems—changes that are not fully accounted for in the CMM. To date, improvement programs for software organizations have often emphasized process or technology, not people. This is the main motivation for the *People Capability Maturity (P-CMM)* model.

The P-CMM describes an evolutionary improvement path from ad hoc, inconsistently performed practices, to a mature, disciplined, and continuously improving development of the knowledge, skills, and motivation of the workforce. The P-CMM helps software organizations:

Table 7: Other Models for Process Improvement

Name of the Model	Purpose	Yr Estd.	The People Behind	Remarks
The Trillium Model—8 Capability Areas: • Organisational • Process Quality - o Human Resource Development and Management • Process • Management • Quality System • Development Practices • Development Environment • Customer Support	Describes a telecommunications product development and support capability To be used in a number of ways: -To benchmark an organization's product development and support process capability against the best practices in the industry -In self-assessment mode, to help identify opportunities for improvement within a product development organization -in pre-contractual negotiations, to assist selecting a supplier	1994	Bell Canada, Northern Telecom and Bell-Northern Research	Can also be used as a reference benchmark in an internal capability improvement program
Bootstrap	Bootstrap Methodology with the following objective to help increase process effectiveness, to support improvement planning etc.	-	Participating partners in the Esprit project	Bootstrap Process Model has the following capability levels: Level 0: Incomplete Process Level 1: Performed Process Level 2: Managed Process Level 3: Established Process Level 4: Predictable Process Level 5: Optimizing Process
Ideal Model	Serves as a roadmap for initiating, planning and implementing improvement actions	-	Software Engineering Institute (SEI)	Ideal has five Phases: Initiating, Diagnosing, Establishing, Acting, Learning

- Characterize the maturity of their workforce practices
- Guide a program of continuous workforce development
- Set priorities for immediate actions
- Integrate workforce development with process improvement
- Establish a culture of software engineering excellence

The P-CMM is designed to guide software organizations in selecting immediate improvement actions based on the current maturity of their workforce practices. The benefit of the P-CMM is in narrowing the scope of improvement activities to those practices that provide the next foundational layer for an organization's continued workforce development. These practices have been chosen from industrial experience as those that have significant impact on individual, team, unit, and organizational performance. The P-CMM includes practices in such areas as:

- ☐ work environment
- ☐ communication
- ☐ staffing
- ☐ managing performance
- ☐ training
- ☐ compensation
- ☐ competency development
- ☐ career development
- ☐ team building
- ☐ culture development

Figure 8 shows the Five Maturity Levels of the P-CMM. Each maturity level provides a layer in the foundation for continuous improvement of an organization's workforce practices. In maturing from the *Initial* to the *Repeatable* level, the organization installs the discipline of performing the basic practices. In maturing to the *Defined* level, these practices are tailored to enhance the particular *knowledge, skills, and work methods* that best support the organization's business. In maturing to the *Managed* teams and empirically evaluates how effectively its workforce practices are meeting objectives. In maturing to the *Optimizing* level, the organization looks continually for innovative ways to improve its workforce capability and to support individuals in their pursuit of *professional excellence*.

Conclusion

There are a number of frameworks available for "process improvement"; some very well known like the SW Capability Maturity Model (SW CMM) and the ISO 9001, while others not so known. This chapter considered most of them.

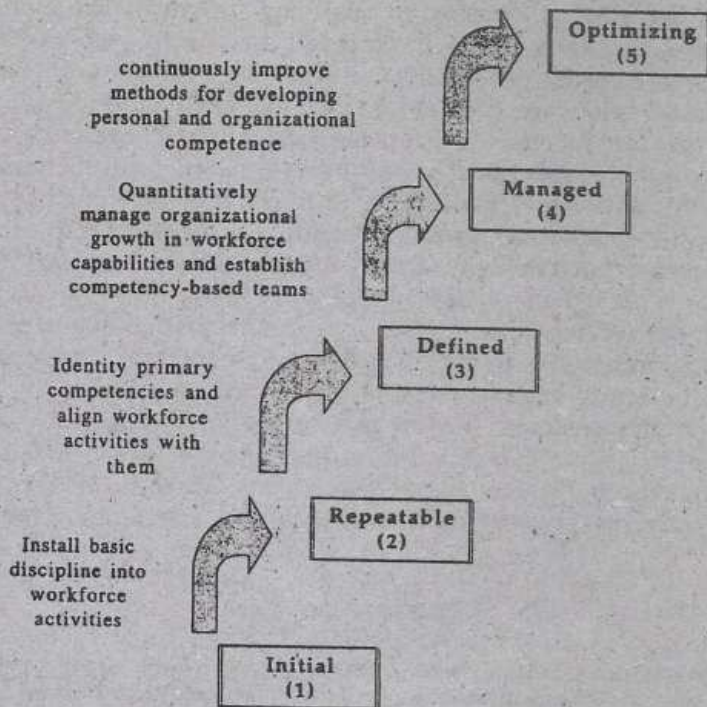


Figure 8 Maturity Levels of P-CMM

The SW-CMM represents a "common sense engineering" approach to software process improvement. Its maturity levels, key process areas, goals, and key practices have been extensively discussed and reviewed within the software community. While the SW-CMM is neither perfect nor comprehensive, it does represent a broad consensus of the software community and is a useful tool for guiding Software Process Improvement efforts.

ISO 9001 has a high acceptance level and that can be traced directly to the simple elegance of its fundamental tenets: Document what you do, Do what you documented and Verify that you are doing it.

One of the objectives of ISO/IEC 15504 is to create a way of measuring process capability, while avoiding a specific approach to improvement such as the SEI's maturity levels, so that the many different kinds of assessment, model, and their results, can be meaningfully compared to one another.

The focus of the P-CMM model is on the maturity of the workforce related practices. Its brief overview is included in this chapter for the sake of completeness of the discussion on improvement frameworks.

It is easy to get overwhelmed by all these process improvement frameworks and models. But never forget why process improvement is important. Standards and models can help organizations improve their

software process, but focusing on achieving a maturity level without really improving the underlying process is dangerous. Maturity levels should be a measure of improvement, not the goal of improvement. Important questions are: What are the business needs and business goals of the improvement effort? What is the impact on cost, cycle time, productivity, quality, and—most importantly—customer satisfaction?

Surveys show that “high maturity organizations” effectively address people issues; they have quality culture and process orientations. Also it turns out that in high maturity organizations the ISO 9001 certification has played a significant role in sustaining the drive towards process improvement. These organizations have also shown to be supporting commitment to Total Quality Management (TQM). Surveys also show that it is very difficult to be a high maturity supplier if you have a low maturity customer (or low maturity suppliers or partners—in the case of strategic alliances, joint ventures, and virtual organizations). In such situations, “educating the customer” becomes a crucial task for shaping customer expectations and getting co-operation from them.

REFERENCES

- [1] *SEI Special Report CMU/SEI-94-SR-13 ESC-SR-94-013* (September 1994) by James Hebsleb, Anita Carelton, James Rozum, Jane Siegel and David Zubrow
- [2] *The Capability Maturity Model: Guidelines for Improving the Software Process* by Carnegie Mellon University
- [3] *Software Process Improvement: Practical Guide for Business Success* by Sami Zahran (Addison-Wesley)
- [4] *CMM Implementation Guide* by Kim Caputo (Addison-Wesley)
- [5] *Introduction to the Personal Software Process* by Watts S. Humphrey (Addison Wesley)
- [6] *Introduction to the Team Software Process* by Watts S. Humphrey (Addison-Wesley)
- [7] *HOW ISO 9001 COMPARES WITH THE CMM*—Marc C. Paulk (Software Engineering Institute 1995)
- [8] *Analyzing the Conceptual Relationship Between ISO/IEC 15504 (Software Process Assessment) and the Capability Maturity Model for Software*—Mark C. Paulk (Software Engineering Institute)
- [9] *Practices of High Maturity Organizations*—Mark C. Paulk Software Engineering Institute (1999 SEPG Conference, Atlanta, Georgia, 8-11 March 1999)
- [10] www.sei.cmu.edu/cmmi/adoption/adoption.html for a number of interesting articles on CMM-Integrated Model
- [11] *Choosing a CMMI Model Representation*—Sandy Shrum Software Engineering Institute
- [12] *Transitioning Your Organization from Software CMM Version 1.1 to CMM-I SW Version 1.0*—A CMM-I White Paper at www.sei.cmu.edu/cmmi/publications/white-paper.html
- [13] *Overview of the People Capability Maturity Model*—Bill Curtis, William E. Hefley Sally Miller (published by Software Engineering Institute at Carnegie Mellon University)

SOFTWARE TESTING

10.1 Overview

Testing is an organizational responsibility. The focus of this chapter is on some important aspects of software testing. Delivering quality products is still a problem in the IT (Information Technology) industry. Testing is not a solution for this. Testing is just one instrument that can contribute to the quality enhancement of the IT systems/software products. Testing should be embedded in the overall quality management of an organization. As such, *Testing Policy* should be defined and documented and *Testing Strategy* should be evolved.

Often, quality assurance gets confused with testing. Sometimes people combine both terms to mean "QA". This is not correct.

- The goal of software tester is to find bugs and find them as early as possible and notify the same to developers so that they get fixed.
- A software quality assurance person's main responsibility is create and enforce standards and methods to improve the development process/testing process so that bugs can be prevented.

Thus, QA is separate from QC and deals with the overall management of quality. At times, due to budget resource constraints a slight overlap can happen between these two roles. Confusion among the team members about who is testing and who is not has caused lots of "process pain" in many projects. It must be remembered that essentially, (software) testing is a quality control (QC) activity. In Chapter 1 we discussed the difference between QA (Quality Assurance) and QC (Quality Control).

In this chapter, Software Testing as a topic is discussed for the completeness of discussion on quality, which is the theme of this book. The following aspects with regard to software testing are covered:

- Purpose of Testing
- Inspection versus Testing
- Testing versus Debugging
- Testing Life Cycle
- Roles and Responsibilities involved in Testing
- The Test Plan
- The V Model for Testing
- Testing Techniques (Testing of Functional and Non-functional attributes)
- Test Metrics
- Risk Based Testing
- Test Automation and Test Tool Selection
- Extreme Testing concept
- Human Issues and Challenges in Testing
- TPI (Test Process Improvement and Test Maturity) Framework
- Software Testing Careers and Testing Competence through professional certifications

Before proceeding further, two important terms: *Verification* and *Validation* are explained below.

Verification and Validation

Verification ensures that various work products (requirement specification document, design document code, test plan etc) are complete and consistent with respect to the other related work products and customer needs

Validation is concerned with assessing the quality of a software system in its actual operating environment. As per IEEE, Validation is the process of evaluating the software at the end of software development process to determine compliance with the user requirements.

Verification: Are we building the product right?

Validation: Are we building the right product?

High-quality cannot be achieved through testing of source code alone and so reviews are important (various review techniques were discussed in Chapter 6)

10.2 Purpose of Testing

Very simply, testing is an activity associated with any process that produces a work product. It is used to determine the status of the product during and after the build. Role of testing changes as the type of process used to build the product changes. Basically, testing is the

process of executing a software program/segment of code with the intent of finding errors. This implies that testing is a "destructive process". From a practitioner's view point, testing has the following roles:

Primary Role

- Determine whether system/software application meets specification (VERIFICATION)
- Determine whether system/software application meets users' needs (VALIDATION)

Secondary Role

- Instill confidence about the product developed
- Provide insight into the software development process
- Continuously improve the testing process

The IEEE definition of testing is: *Testing is the process of exercising or evaluating a system of system component by manual or automated means to verify that it satisfies specified requirements.*

Industry experience shows that testing consumes at least half of the effort expended in producing a working program. In terms of "Cost of Quality" (which comprises of Prevention, Appraisal and Failure), Testing is the appraisal effort. Not many programmers like testing and even fewer like *test design*—especially if test design and testing take longer than program design and coding. There is another problem with testing and it is related to why we do it. It's done to catch bugs. There is a myth that if we were really good at programming, there would be no bugs to catch. Boris Beizer in his famous book [6] provides a very interesting philosophical discussion on this point in terms of the "myth" just mentioned. He says:

".....There are bugs, because we are bad at what we do; and if we are bad at it, we should feel guilty about it. Therefore, testing and test design amount an admission of failure, which instills a goodly dose of guilt. And the tedium of testing is just punishment of our errors. Punishment for what?—For being human? Guilt for what?—For not achieving inhuman perfection?....."

Testing should focus on bug identification. To the extent that testing and test design do not prevent bugs, they should be able to discover symptoms caused by bugs. Finally, test should provide clear diagnoses so that bugs can be easily corrected. Bug prevention is the final goal based on analysis of past defect data collected by the testing process. A prevented bug is better than a detected and corrected bug because if the bug is prevented, there is no code to correct. Moreover, no

re-testing is needed to confirm that the correction was valid, no one is embarrassed, no memory is consumed. More than testing, designing effective test cases is one of the best prevention actions.

The most important considerations in testing, according to Myers [11] are psychological in nature. He puts forth a set of vital testing principles or guidelines, which are summarized below:

1. A necessary part of a test case is a definition of the expected output or result
 2. A programmer should avoid attempting to test his/her own program
 3. A programming organization should not test its own programs
 4. Thoroughly inspect the results of each test
 5. Test cases must be written for invalid and unexpected, as well as valid and expected, input conditions
 6. Examining a program to see if it does not do what it is supposed to do is only half of the battle. The other half is seeing whether the program does what it is not supposed to do
 7. Avoid throw-away test cases unless the program is truly a throw-away program
 8. Do not plan a testing effort under the tacit assumption that no errors will be found
 9. The probability of the existence of more errors in a section of a program is proportional to the number of errors already found in that section
 10. Testing is an extremely creative and intellectual challenging task
 11. Testing is the process of executing a program with the intent of finding errors
- A good test case is one that has a high probability of detecting an as-yet undiscovered error
- A successful test case is one that detects an as-yet undiscovered error

10.3 Differences Between Inspection and Testing

Having discussed the purpose of testing, now let us understand the difference between Testing and Inspection, which is a typically asked question. In Chapter 6 we discussed *Reviews* and *Inspection*. We discussed there that *Verification* is a "human" examination or review of the work product. There are various types of reviews as discussed in Chapter 6 (Inspections, Walkthrough, Technical Reviews).

Inspection can be used long before executable code is available to run tests. Thus, *Inspection* can be applied much earlier than testing the executable code and can be used to "verify" the requirements as well as the design. Tests to be run can be defined along with requirements and design specification process. This is because specification is the source for knowing the expected result of a test execution.

The one thing, which Testing does and Inspection does not, is to evaluate the software while it is actually performing its function in its intended/simulated environment. Inspection can only examine static documents and models; testing can evaluate the product working. To this extent, it can be said that Inspection is a *static testing* technique. The figure below shows the relationship between Testing and Inspection.

Inspection (among other verification techniques) is concerned with removal of any possible defects that may exist prior to the coding phase. The information gained from defects found by running test could be used in the same way, but this is rare in practice.

It should be noted that *Testing* and *Inspection* are NOT mutually exclusive alternatives. Test documentation is just as prone to defects as any other document, so it is essential that test documents be inspected. Inspecting the *Test Plan* can help bring out potential defects in the test plan itself which is the basic guiding document to execute tests. Thus, both *Inspection* and *Testing* aim towards establishing confidence in the work product. Both have different but complementary objectives and must be used appropriately to achieve maximum benefit.

Inspection

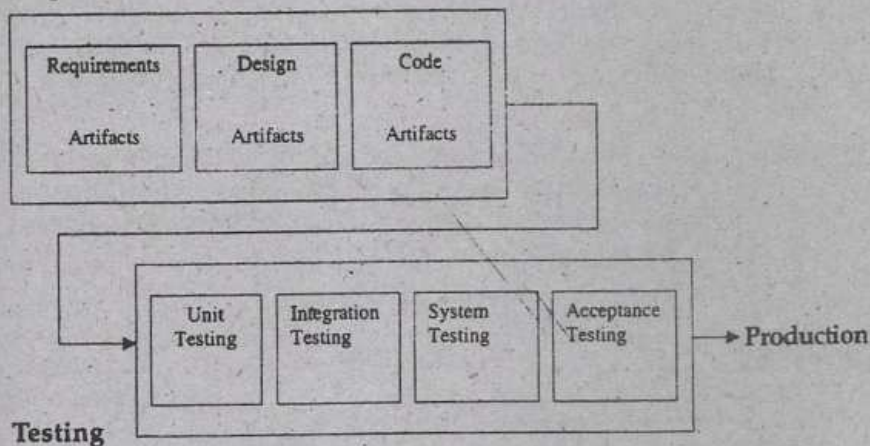


Figure 1: Testing vis-à-vis Inspection

Professional software testing practices include:

- The planning and design of testing from the beginning of the software development life cycle (Life Cycle Testing concept)
- Acceptance tests criteria defined by users at the same time as user requirements are defined
- Systems test cases identified during requirements analysis—integration test cases identified during architectural design
- Unit tests cases identified during detailed design

- Execution of unit tests, integration tests, system tests, acceptance tests
- Evaluation of test effectiveness

The advantage of early test planning is that Inspection can be effectively used to improve the quality of the test planning documents and the test planning process at early stages of a project. This will help avoid "last minute before deadline" problems in pushing the deliverables successfully out of the door.

In summary, we should note that Inspection *does not* replace Testing. They both perform some unique functions, which neither can replace for the other. What Testing *could* find later, Inspection could find *earlier*, and *probably* provides saving in correction effort. Defects injected *after* Inspection must also be found during Testing. Thus, Inspection, in a way, is form of (static) Testing and is a human intensive activity in the sense that automation is not used here. It should be noted that Inspection and Testing in combination is typically more effective than either of these techniques alone.

10.4 Testing v/s Debugging

Often these two terms are not clearly segregated. According to Boris Beizer [6] there is an attitudinal progression in differentiating between Testing and Debugging. This is characterized by the following five phases:

PHASE 0—There's no difference between testing and debugging other than in support of debugging, testing has no purpose. Actually, there is a lot of difference between Testing and Debugging as shown in table 1 below.

Table 1: Testing and Debugging

Testing	Debugging
1. Starts with known conditions, user predefined procedures, predictable outcomes	Unknown initial conditions, end cannot be predicted
2. Should be planned, designed, scheduled	Cannot be constrained
3. Is a demonstration of error/apparent correctness	Is a deductive process
4. Proves a programmer's "failure"	Vindicates a programmer
5. Should strive to be predictable, dull, constrained, rigid, inhuman	Demands intuitive leaps, conjectures, experimentation, freedom
6. Much can be done without design knowledge	Impossible without design knowledge
7. Can be done by outsider	Must be done by insider
8. Theory of testing is available	Only recently have theorists started looking at it
9. Much of test design and execution can be automated	Automation is still a dream

PHASE 1—The purpose of testing is to show that the software works.

PHASE 2—The purpose of testing is to show that the software doesn't work.

PHASE 3—The purpose of testing is not to prove anything, but to reduce the perceived risk of not working to an acceptable value ("Risk-based Approach to Testing")

PHASE 4—Testing is not an act. It is a mental discipline that results in low-risk software without testing effort.

The debugging process is depicted in Figure 2 below in reference to Table 1 above. A point to ponder is that there appears to be some evidence that "debugging prowess" is an innate human trait; some individuals are good at it, others are not. In fact, large variances in debugging ability have been reported for programmers with the same educational experience background.

Debugging Process

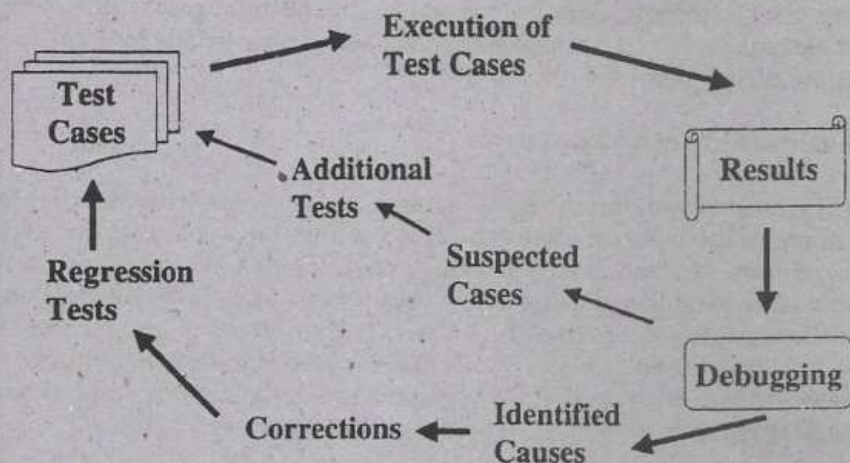


Figure 2: Debugging Process

In the words of Scheiderman:

Debugging is one the more frustrating parts of programming. It has elements of "problem solving" or "brain teasers", coupled with the annoying recognition that you've made a mistake. Heightened anxiety, and the unwillingness to accept the possibility of errors, increases the task difficulty. Fortunately, there is a great sigh of relief and a lessening of tension when the bug is ultimately corrected.....

With this backdrop, we'll discuss "human issues and challenges" in testing, towards the end of this chapter. For now, let us turn to understand the testing life cycle.

10.5 Testing Life Cycle

Successful completion of work product verification activities through review techniques (such as Walkthroughs and Inspections as discussed in section 10.2 of this chapter and Chapter 6) should be the only approved gateway to proceed to the next phase of activities. Figure 3 depicts typical quality gates that apply during the *testing life cycle*.

It shows the testing life cycle sequence starting with the decision to enter the test phase and quality gates before proceeding to the next phase. It should be noted that quality gates exist throughout the entire project life cycle.

The test team needs to verify that the output of any one stage represented in Figure 3 is fit to be used as the input for the next stage. Verification that output is satisfactory may be an iterative process, and this goal is accomplished by comparing the output against applicable standards and project specific criteria. Let us now briefly look at each of the phase mentioned in Figure 3.

1. Decision to enter Test Phase

The results of walkthroughs, inspections, and other test activities should adhere to the levels of quality outlined within the applicable software development standards. Test team's compliance with these standard should be established using the various review techniques (Inspection, Walkthroughs, and Formal Technical Reviews etc. as discussed earlier in this chapter and in Chapter 6). This together with involvement of the test team early in the development life cycle improves the likelihood that the benefit of defect prevention will be realized.

2. Introducing the Test Process

This is formal announcement to the project team that the test phase is now to commence. Roles and Responsibilities are established at this stage and expectations set accordingly. Section 10.6 has a detailed discussion on this.

3. Test Plan Preparation

Test plan is the basic guiding document to carry out the testing phase. You need this to successfully manage the testing project. Writing a test plan allows you to collect your thoughts, your ideas and memories. In short, writing a thorough test plan gives you a chance to crystallize

your testing knowledge and experience into a concrete way of tackling the test tasks ahead. It is your opportunity to communicate with your test team. Section 10.8 has a discussion on the *test plan*.

4. Test Case Development

This is an extremely important activity whose roots lie in the requirements analysis itself. Aim should be to develop re-usable test cases, which could save a lot of efforts in the next similar project. Careful and methodical planning for test cases is important for their organization, repeatability, tracking and generating proof of testing.

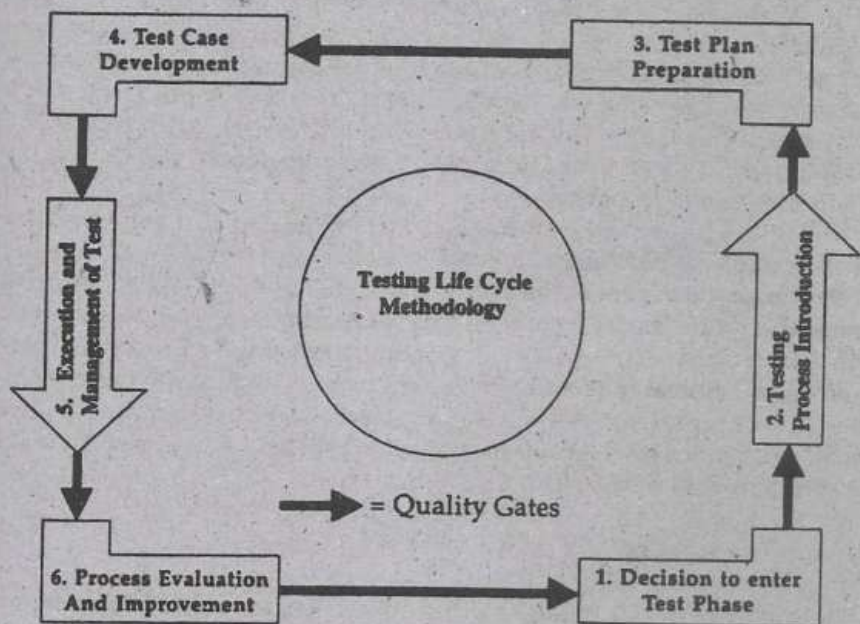


Figure 3: Testing Life Cycle and Quality Gates

5. Executing and Managing the Test Plan

The main activities involved during this are assigning the test cases to testers, gathering the test results, collecting the test process metrics (section 10.11), preparing bug reports etc.

6. Test Process Evaluation and Improvement

Testing belongs to the detective measures of a quality system. At the end of the testing phase, it is important to analyze the effectiveness of the testing process and introduce improvement measures as required. The purpose of improvement is to be able to carry out the process with

less effort and budget next time (In section 10.11 we have discussed some test metrics that help improve the testing process)

10.6 Roles and Responsibilities in Testing

This section describes the roles involved in the testing process, their individual responsibilities, and their requirements for reporting and liaison. It is important to note when reading this section that it is possible for a single person to assume more than one role within any testing process. The number of roles simultaneously existing would depend on the size and scope of the testing activity. For example, in a small organization in which there are a few testing professionals, one individual may assume the roles of both *Test Analyst* and *Tester* on a testing project. Similarly, one person may assume the responsibilities of the *Testing Manager* and *Test Team Leader* if there is only one testing task or project to manage. The only exception to this is the *Independent Test Observer*, who must maintain complete independence from the testing activities and its progress.

A single person may also assume different roles on different projects. For example, a *Test Analyst* on one project may perform the role of *Test Team Leader* on another. Again, this will depend on the size of the business and number of test professionals involved in testing. Later in this section, we also discuss some supplementary testing roles. These roles may be beneficial to include in testing projects with particular characteristics or requirements—such as a project in which automated testing tools are employed (Test automation and Test Tool selection is discussed in section 10.13 of this chapter)

10.6.1 Test Manager

This role should be created right from the beginning of the software development project and certainly not later than when the requirements gathering phase is over. The *Test Manager* has the responsibility to administer the organizational aspects of the testing process on a day-to-day basis and is responsible for ensuring that the individual testing projects produce the required products (i.e. the outputs from the testing phases, including the fully tested AUT—application under test) to the required standard of quality and within the specified constraints of time, resources and budget.

Where appropriate, the *Test Manager* is also responsible for liaison with the development teams to ensure that they follow the Unit Testing and Independent Testing approach documented within the process. The *Test Manager* will also liaise with the *Independent Test Observer(s)* to receive reports on testing activities that have failed to follow the testing process appropriately.

The Test Manager reports to a senior management or director within the organization, such as the *Quality Assurance (QA) Manager* or *Information Technology Director*. In large organizations, and particularly those following a formal project management process, the Test Manager may also report to a Testing Program Board, which is responsible for the overall direction of the project management of the testing program.

10.6.2 Test Team Leader

In small testing projects the Test Manager himself may play this role. In larger projects this role needs to be created separately. This role should be in place after the test plan is ready and test environment is established. The **Test Team Leader** is given the responsibility to carry out the day-to-day testing activities. His or her responsibilities assigning tasks to one or more testers/test analysts, monitoring their progress against agreed upon plans, setting up and maintaining the testing project filing system, and ensuring the generation of the test artifacts as described in section 10.7.

One or more *Test Analysts* and *Testers* report to the Test Team Leader, who, in turn, reports to the Test Manager. The Test Team Leader will liaise with the Independent Test Observer (to discuss availability to attend a particular test, for example), and, where appropriate, to the *Development Team Leader* (to undertake early planning of the testing project and preliminary test design and to determine the availability of the AUT for testing). During *Acceptance Testing* the *Test Team Leader* is responsible for liaising with the User Representative and Operations Representative to obtain one or more users to perform *Acceptance Testing*, which is the responsibility of the customer.

10.6.3 Test Analyst

The *Test Analyst* is responsible for the design and implementation of one or more *Test Scripts* (See Table 3 for the format), which will be used to accomplish the testing of the AUT. It should be noted that if the scope of testing activity is small, the test lead might end up doing many of tasks described here. The *Test Analyst* may also be called upon to assist the *Test Team Leader* in the generation of the *Test Specification Document*. During the design of *Test Case* (see section 10.7 on *Test Artifacts*), the Test Analyst will need to analyze the *Requirements Specification* for the AUT to identify specific requirements that must be tested. During this process the Test Analyst will need to prioritize the Test Cases to reflect the importance of the feature being validated and the risk of the feature failing during normal use of the AUT. At the completion of the

testing project, the Test Analyst is responsible for the back-up and archival of all testing documentation and materials, as well as the creation of a testing re-use pack. The Test Analyst is also responsible for completing a Test Summary Report briefly describing the key points of the testing project.

10.6.4 Tester

The *Tester* is primarily responsible for the execution of the Test Scripts created by the Test Analyst and for the interpretation and documentation of the results of the Test Case Execution (Table 4 shows the format for a *Test Script*). During the test execution, the *Tester* is responsible for filling in the Test Result Record Forms (Table 4) to document the observed result of executing each Test Script (Table 3) and for co-signing the bottom of each form with the *Independent Test Observer* to confirm that the Test Script was followed correctly and the observed result recorded accurately. The *Tester* is also responsible for the recovery of the test environment in the event of failure of the system.

10.6.5 Independent Test Observer

In general terms, the *Independent Test Observer* is responsible for providing independent verification that correct procedure (as per the testing process for the organization) are followed during the testing of the AUT. Sometimes this can be contractual requirement stipulated by the customer. Specifically during testing, the *Independent Test Observer* is responsible for ensuring that the *Tester* executes the tests according to the instructions provided in the *Test Scripts* (Table 3 has the format) and that he or she interprets the observed result correctly according to the description of the expected result. In organizations in which there is a formal *Quality Assurance Group*, *Independent Test Observers* may be drawn from the ranks of the Quality Assurance Representative. In smaller organizations or where there is not such a strong quality culture, the *Independent Test Observer* may be a staff member drawn from another group or project within the organization. The key criterion in selecting a *Independent Test Observer* is that she or he must be impartial and objective.

10.6.6 Supplementary Testing Roles

As said earlier, depending on the specific testing requirements of a particular testing project, it may be appropriate to include a number of additional testing roles. These are summarized below:

Table 2: Supplementary Testing Roles

Name of the Supplementary Role	Responsibilities
TEST AUTOMATION ARCHITECT	• Definition of the test automation architecture • Specification of all hardware and software configurations added • Assisting the <i>Test Team Leader</i> to plan any programming and set-up tasks needed to install a new automation environment, including deploying the testing tools and environment-specific support code.
TEST AUTOMATION ANALYST	• Implementing automated tests as specified in the <i>Test Specification Document</i> and any supplementary design documents • Implementing test support code required for efficiently executing automated tests • Setting up and executing automated tests against the AUT • Reviewing the automated test log following testing to identify and report on defects found within the AUT during testing
EXPLORATORY TESTER	• Autonomous testing of a product without following a specified set of Test Cases • Focusing on finding the highest severity defects in the most important features of the system using a risk-based testing approach

10.7 Test Artifacts

In section 10.6.2 we mentioned that the *Test Team Leader* is responsible for the various test artifacts. Test artifacts are work products that get created as part of numerous activities carried out during the testing phase of a software development project. In this section we have provided brief description of the test artifacts. IEEE definitions are also provided for some important terms in the context of testing. In addition to this, we have also provided some useful formats (Table 3: Test Script, Table 4: Test Results/Execution Table, Table 5: Test Plan Template).

- *The Test Plan Document*, which will be used as the basis of project management control throughout the testing process. (The Test Plan will be discussed in a separate section in this chapter). This document is produced by the *Test Team Leader* and is approved by the *Testing Manager*
- *The Test Specification Document*, which outlines in detail the approach to testing the AUT (Application Under Test), the resources needed, the testing environment, the evaluation criteria and acceptable defect rates /frequencies, exit criteria, and so on. At the minimum, the test specification document should include the following:

- o Introduction
 - Background
 - Scope
 - Structure of the Document
 - References
- o Test Requirement
 - Introduction
 - Test Philosophy (Overview, Functional Areas, Test Result Categories, Exclusions)
- o Test Environment
 - Overview
 - Hardware
 - Software
 - Test Data
- o Staff Roles and Responsibilities
- o Test Identification
 - Test Scripts
 - Result Reporting
 - Acceptance Criteria
 - Test Error Clearance
 - Test Documentation
- o Test Procedure
 - Introduction
 - Pre-Test Activities
 - Concluding the Test
 - Post-Test Activities
- o Test Case
- o Test Suite
- o Test Harness
- o Test Bed

Some IEEE Definitions (IEEE Std. 610.12)

Test

An activity in which a system or component is executed under specified conditions, the results are observed or recorded, and an evaluation is made of some aspect of the system/component

Test Documentation

Documentation describing plans for, or results of the testing of a system or components

Test Bed

An environment containing the hardware, instrumentation, simulation, software tools, simulators and other support elements needed to conduct a test

Test Case

A set of test inputs, execution conditions, and expected results, developed for a particular objective such as to exercise a particular program, path or to verify compliance with a specific requirement

Test Coverage

The degree or extent to which a given component must meet in order to pass a given test.

Test Driver (synonymous with Test Harness)

A software module used to invoke a module under test and often, to provide test inputs, control and monitor execution, and report test results.

Test Case Generator

A software tool that accepts as input source code, test criteria, specification, or data structure definitions; uses these inputs to generate test input data, and sometimes determines expected results (synonymous with *Test Data Generator*, *Test Generator*)

Test Case Specification (synonymous with Test Description, Test Specification)

A document that specifies the test inputs, execution conditions, and predicted results for an item to be tested.

Test Report

A document that describes the conduct and results of the testing carried out for a system or component.

Acceptance Criteria

The criteria that a system/component must satisfy in order to be accepted by a user, customer or other authorized entity.

Table 3: Test Script

Project ID	Version Test Date	Script Author
AUT Title		
Testing Phase		
Test ID		
Purpose of Test		
Test Environment		
Test Steps		
Expected Results		

Table 4: Test Result Record

Project ID	Effort Spent (Hrs)
AUT Title	AUT Version
Testing Phase	Test Date
Test Case Id	Tester Name
Testing Cycle No	
Test ID	
Test Execution Steps	
Actual Test Results Observed	
Defect Id *	
Test Result Category **	
Test Error Description	
Tester Signature	
Reviewer Signature	

* This gets logged into the Defect Tracking System

** Test Result Categories (indicated on the next page)

Test Result Categories (Ref. Table 3)

PASS	observed test result conforms to the expected result
FAIL	observed test result does not conform to the expected result at all
ACCEPTABLE	observed test result indicates that the system differs from the agreed specification but is acceptable, requiring no change to the AUT, but requiring a change to the Functional Specification
TOLERABLE	observed result is incorrect; the AUT is workable and will be accepted but the defect must be rectified within an agreed-on time period
INTOLRABLE	observed result is incorrect and the defect must be corrected before the AUT passes this testing phase (Quality Gate)
TEST ERROR	observed result is correct, but the expected result is incorrect (e.g. a typographical error in the test script)
BREAK	test stopped because of some reasons

10.8 The Test Plan

Given that exhaustive testing is impractical, except in the most critical of the systems, the project team must design and plan a testing strategy (commonly known testing strategies are *White Box Testing* and *Black Box Testing* although these days with Web Applications a third term called "*Gray Box Testing*" is also heard) that utilizes a balance of testing techniques to cover a representative sample of the system. The test planning process is a critical step in the testing process. Without a documented test plan, the test itself cannot be verified, coverage cannot be analyzed, and the test process is not repeatable.

In section 10.7, we have provided definitions of common test terminology. Reader should keep this in mind while reading this section. A document that defines the overall testing objectives and approach is called a *Test Plan* while a document that defines what is selected to test, and describes the expected results is called a test design. Test plans and designs can be developed for any level of testing, and may be combined in the same document.

Test Planning should begin at the same time requirements definition starts. The plan will be detailed in parallel with application requirements. During the analysis stage of the project, the Test Plan defines and communicates test requirements and the amount of testing needed so that accurate test estimates can be made and incorporated into the test plan.

Why write a Test Plan? Writing a test plan allows you to collect your thoughts, your ideas, and your memories. Writing a thorough test plan gives you a chance to crystallize that knowledge into a concrete way to tackling the tasks ahead. Basically, a test plan is also an

opportunity to communicate with your test team, your development team and your project management.

Thus, documented tests are **REPEATABLE**, **CONTROLLABLE**, and ensure **ADEQUATE TEST COVERAGE** when executed.

Good testing requires thinking out an overall approach, designing tests, and establishing expected results for each of the chosen *test cases*. Since exhaustive testing is usually impractical, the test team must select a sample of tests that will provide the appropriate confidence level in the application. The planning and care extended on marking this selection and planning it, is the difference between a good test process and a poor one.

There are standards in the industry that define hierarchies of test documentation. In section 10.7, we had discussed a few key terms as defined in the IEEE standard. Here, we have repeated some terms for quick handy reference to the reader.

- The *Test Plan* is defined as an overall document providing direction for all testing activity
- The *Test Design Specification* refines the test approach and identifies the features to be covered by the design and its associated tests
- A *Test Case Specification* documents the actual values used for input along with the anticipated outputs
- The *Test Procedures Specification* identifies all steps required to exercise the specified test cases

The standard also addresses documents to be used in test reporting. Test plans and formats vary from organization to organization. A typical Table of Contents of a test plan is indicated below:

Table 5: Test Plan Template

Overview
Bounds
Scope and Objectives
Definitions
Setting
Assumptions
Quality Risks
Proposed Schedule and Milestones
Transitions
Entry Criteria
Discontinuation Criteria
Exist Criteria
Test Configurations and Environment
Test Design and Test Tools (Automation Strategy)
Test Execution
Resources
Tracking and Management of Tests and Bugs
Bug Isolation and Classification
Release Management
Test Cycles
Test Data Management
Risks and Contingencies
Change History
Referenced Documents

10.9 The V-Model for Testing Phases

The V-Model shown in figure 4 is software development and testing model, which helps to highlight the need to plan and prepare for testing early in the development process. The left-hand descending arm of the "V" represents the traditional Waterfall development phases, whereas the ascending right-hand arm of the "V" shows the corresponding testing phase. In the V-Model, each development phase is linked to a corresponding testing phase. We'll discuss each of these briefly in this section. As work proceeds on a particular development phase, planning and some preliminary design work is performed for the corresponding testing phase.

The benefits of the V-Model are the following:

- The testing phases are given the same level of management attention and commitment as the corresponding development phases in terms of planning and resourcing. This allows risks to be addressed early in the development cycle
- The outputs from the development phases (such as specification and design documents) can be reviewed by the testing team to ensure their testability
- The early planning and preliminary design of tests provides additional review comments on the outputs from the development phase

We'll now discuss each of the testing phases briefly as shown in the right hand arm of the V-Model depicted in Figure 4.

10.9.1 Unit Testing

Unit Testing is also referred to as "software component testing". It represents the lowest level of testing that the AUT undergo. It is conducted to ensure that reliable program units (software components) are produced that meet their requirements. Unit testing is typically conducted by the *Development Team* and specifically by the programmer who coded the unit, the person responsible for designing and running a series of tests to ensure that the unit meets its requirements. It is unlikely that the Development Team Leader will need to produce detailed Test Plan and Test Specification documents for Unit Testing. The results of Unit Testing should be formally documented using a Test Result Record Form (such as the one shown in Table 4 in the earlier section).

10.9.2 Integration Testing

The objective here is to demonstrate that the software modules, which comprise the AUT interface, interact in a correct, stable and coherent manner. The Development Team typically conducts Integration

Testing, with responsibility for managing the testing resting with the Development Team Leader. In large organizations with well-established quality and/or mature software development and testing process, Integration Testing is conducted by a dedicated Testing Team. This team typically comprises of development team members and testing team members. The team is responsible for all aspects of testing. The results of Integration Testing should be formally documented using a suitable form for Test Result Record, which is filed by the Development Team Leader at the end of the testing phase. Another key issue to consider is the need to manage the user expectations regarding the look and feel and the operation and performance of the AUT.

10.9.3 System Testing

The fundamental objective of *System Testing* is to establish confidence that the AUT will be accepted by the users—i.e. that it will pass its *Acceptance Test*. System Testing is typically conducted as an independent testing project by a dedicated testing team, with responsibility for managing the testing falling to the Test Team Leader. For small organizations, it is possible that the development team will conduct System Testing. During System Testing, it is essential that truly independent observation of the testing process be performed. The *Independent Test Observer* may be drawn from another development project, or from the Quality Assurance group if the organization has one. As with Integration Testing, the Test Team Leader should consider inviting the User

The V Model for Testing

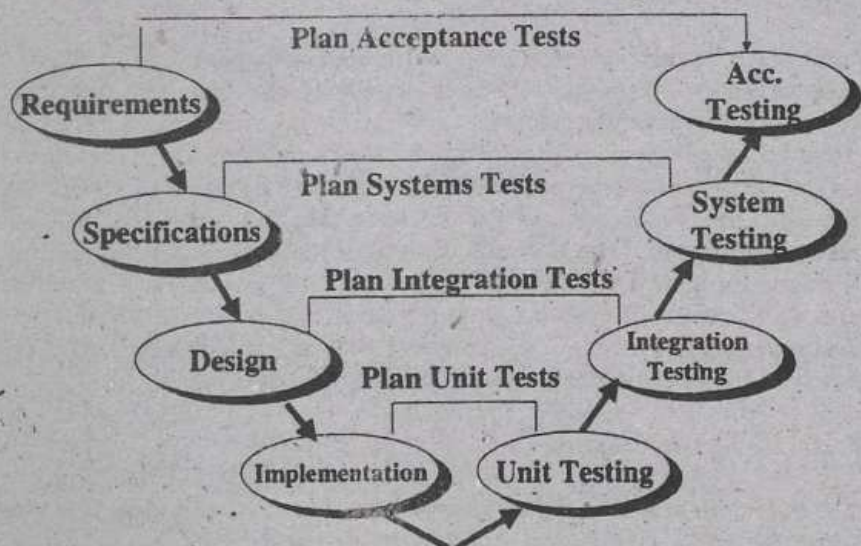


Figure 4

Representative to the System Test in an informal capacity to observe the testing process in order to manage the users' expectations of the AUT prior to formal Acceptance Testing.

10.9.4 Acceptance Testing

The purpose of Acceptance Testing is to confirm that the system meets its business requirements and to provide confidence that the system works correctly and is usable before it is formally "handed over" to the end users. Customer is responsible for ensuring this. Acceptance Testing is sometimes divided into *User Acceptances Testing* (involving the business or end users of the AUT) and *Operations Acceptance Testing* (involving the administrative/support staff of the AUT). Acceptance Testing is performed by nominated user representatives under the guidance and supervision of the testing team. During Acceptance Testing it is very important that independent observation of the testing process be performed, particularly where the user representatives are not so IT-savvy.

10.9.5 Alpha and Beta Testing

Before we move on to the next section, it is important to discuss these two important terms for the sake of completeness. In a way, the two terms are connected with *acceptance testing*. The testing ideas talked so far have been "internal"; in the sense that the people involved are from the developers' organization. Alpha and Beta testing involves "others" (external to the developers' organization) to validate that the software is working as intended.

Alpha Testing

Let us first discuss the scenario for alpha testing. However much a thorough plan may exist with the development team of the supplier organization, the fact remains that it is virtually impossible to "foresee" how the customer will "really" use a program/module/system. Operating manuals and user manual may be written and get reviewed before giving it to the customer as a part of agreed deliverables but still there is always a possibility that instructions for use may be misinterpreted. Also, strange combination of data may happen, business rule may change subtly and the outputs produced actually under operational environmental may not be understood by the end user though the tester may have found it to be ok when he/she checked it.

When custom software is produced for one particular customer, a series of acceptance tests are conducted which helps customer to validate all requirements. Acceptance testing can be conducted over a period of weeks/months so that cumulative errors can be uncovered. If the software is developed as a product to be used with varied types of cus-

tomers (typically this happens with utility software), formal acceptance testing may not be feasible or practical. In such situations, a process called *alpha testing* and *beta testing* is used to uncover errors that are likely to be found only by the end users.

A customer conducts the alpha test at the developer's site. The software is used in its intended operating environment. The developer assumes the role of an "invisible observer" noting and recording the errors/usage problems as the end user interacts with the system/software.

Beta Testing

Now let us understand the scenario for *beta testing*. Beta testing is the term used to describe the "external testing process" wherein the software is sent out to a select group of potential customer who use it in a "real-world environment". Beta test takes place when the developer is confident that the software is ready for the final delivery from the development organization side. Its purpose is to validate that the software is ready to release to "real customers". Some key points about beta testing are given below:

- "Know" your beta testers—since a beta test can have different goals, it is important to understand in advance who is going to participate in it: are they experienced "techies" or novice users? How useful/user-friendly the software is perceived depends really on this. If the beta testers are too advanced users, then they may not be interested in the lower details, which may really concern the novice users. So the selection of people in the best testing activity is important.
- Are you sure that the beta testers really used the software? You may have signed up with hundreds of beta tester and for months they may not report anything. Does that mean that there are "no bugs"? It is important to ensure that there is adequate follow-up happening with the beta testing participants.
- Beta tests can be a good way to find compatibility and configuration bugs; especially with the web-based applications.
- For usability issues, signing up beta testing makes a lot of sense provided the selection of participants is made carefully.
- With beta testing a risk is that only obvious, superficial problems may get reported. To motivate beta testers to spend adequate time on trying out the system, innovative schemes can be designed. If beta testers report the already known bugs to the developers then it does not serve the purpose. It is a good idea to provide a list of already found bugs to the beta testers so that they can focus on other areas.

Finally, in summary, we note that beta test is conducted at one or more customer sites by the end user(s) of the software/system. Unlike alpha testing, the developer is generally not present. Therefore, the beta test is a "live" application of the software in an environment that cannot be controlled by the developer. The customer records all the problems encountered and is expected to report these to the developer. This interaction of reporting problems at regular intervals may continue for quite some time. Based on the reported problems, the developer makes modifications and then prepares the software for further releases (by giving "fixes" and "patches" etc.) for the entire customer base.

10.10 Testing Techniques

Broadly speaking testing techniques can be divided into two major categories: Functional Testing and Non-Functional Testing.

[A] **Functional Testing Techniques** such as:

- Equivalence Partitioning
- Boundary Value Analysis
- Intrusive Testing
- Random Testing
- State Transition Analysis
- Static Testing
- Thread Testing

[B] **Non-functional Testing Techniques** such as:

- Configuration/Installation Testing
- Compatibility and Interoperability Testing
- Documentation and Help Testing
- Graphical User Interface (GUI) Testing
- Fault Recovery Testing
- Performance Testing
- Reliability Testing
- Security Testing
- Stress Testing
- Load/Volume Testing
- Usability Testing

We'll discuss each of these only very briefly in the following sections.

10.1 Functional Testing Techniques

Equivalence Partitioning

Selecting test data and designing "appropriate" test cases is the single most important task that software testers do and equivalence partitioning (also

known as equivalence classing), is the means by which this is done. Basically, an equivalence class or equivalence partition is a set of test cases that tests the same thing or reveals the same bug. Equivalence Partitioning relies on the fact that inputs and outputs from the AUT can be grouped or partitioned into coherent groups or classes and that all instances of these classes will be treated in the same manner by the system or application under test. The principal assumption of this technique is that testing one instance of the class is equivalent to testing all of them. The skill in using this technique lies in identifying the partition of values and selecting representative values from within this partition, and in deciding how many values to test.

Steps in Equivalence Partitioning are:

- Identify all inputs
- Identify all outputs
- Identify equivalence classes for each input
- Identify equivalence classes for each output
- Ensure that test cases test each input and output equivalence class at least once

As an example of Equivalence Partitioning technique, consider the following:

Example

A subroutine takes in Marks, Student Type and returns the Grades for the student in that subject

Inputs are:

Marks : (0-100)

Student Type : First time/Repeat

Outputs are:

Grade : F,D,C,B,A

Rules

Student Type First Time

0-40 = F, 41-50 = D, 51-60 = C, 61-70 = B, 71-100 = A

Student Type Repeat

0-50 = F, 51-60 = D, 61-70 = C, 71-80 = B, 81-100 = A

Some Test Cases to be considered:

Marks 30 and First Time Student (Expected Output is 'F')

Marks 40 and Repeat Student (Expected Output is 'F')

Marks 65 and First Time Student (Expected Output is 'B')

Marks 85 and First Time Student (Expected Output is 'A')

Marks 0 and First Time Student (Expected Output is 'F')

Marks 0 and Repeat Student (Expected Output is 'F')

Boundary Value Analysis

The best way to describe the concept behind this is through an analogy: If you can safely and confidently walk along the edge of a cliff without falling off, you can almost certainly walk in the middle of a field! If software can operate on the edge of its capabilities, it will almost certainly operate well under normal conditions.

Boundary (Value) Analysis is a technique related to Equivalence Partitioning and relies on the same underlying principal: that inputs and outputs can be grouped into classes and all instances of these classes can be treated similarly by the system. Whereas Equivalence Partitioning deals with selecting representative values from within the class, Boundary Value Analysis forces on the testing of values from the boundary of the class. Specifically, the technique will design a Test Case for the boundary value itself, plus a Test data for one significant value on either side of the boundary.

Thus an additional step involved for constructing test cases based on Boundary Value Analysis is that *for each input equivalent class, ensure that test cases include:*

- One Interior Point
- One Extreme Point
- All Epsilon Points (close the either end of the range of inputs)

Extending the example in the previous section (Equivalence Partitioning), some Test Cases for Boundary Analysis would be:

- Marks 99 and First Time Student (Expected Output = 'A')
- Marks 101 and First Time Student (Expected Output should be a suitable 'Error Msg')
- Marks negative and either First Time Student or Repeat Student (Expected Outcome should be a suitable 'Error Msg')

In conclusion, it is to be noted that boundary conditions are special because programming, by its nature, is susceptible to problems at its edges.

Intrusive Testing

As the name suggests, Intrusive Testing involves the deliberate modification of the AUT or its behaviour for the purpose of testing. For example, where the result of a test would not be visible to the tester (such as the modification of the value of a variable that would not be displayed to the user during the execution of the AUT), the Test Analyst may introduce additional statements into the application to display the value of the variable. Other examples include directly setting the values of variables

using symbolic debuggers or deliberately triggering error conditions within the AUT

Clearly, any modifications made to the system for the purpose of testing must not be delivered to the customer/user of the system. Therefore rigorous change control and configuration management is essential. This technique should be used with extreme caution because there are many examples of systems that have been released containing changes introduced during Intrusive Testing that have manifested themselves during normal operation of the system.

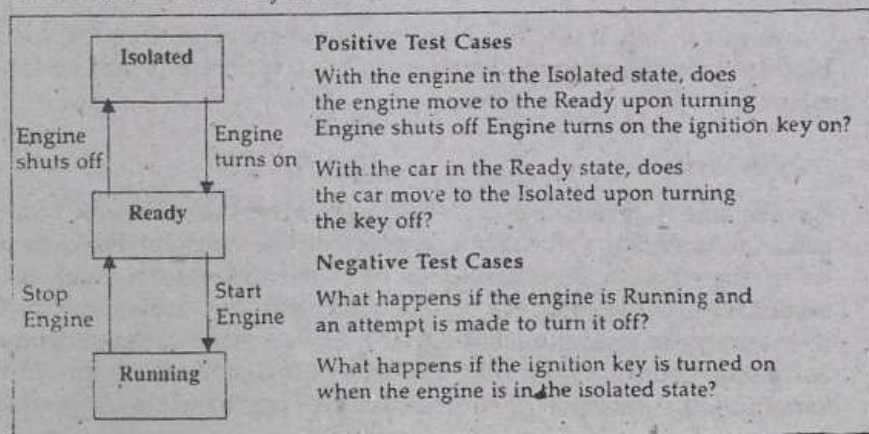
Random Testing

This is one of the very few techniques for automatically generating Test Cases. This is because, although automating the process of providing inputs and stimulating the AUT is relatively straightforward, automating the process of checking the outputs is difficult. Depending on the approach employed, it may also be difficult to reproduce the results of a particular series of random tests. As a result of all this, it is not a commonly used technique.

State Transition Analysis

This is in terms of the states the system can be in, the transition between those states, the actions that cause the transitions, and the actions that may result from the transitions. For designing Test Cases, State Transition information can be derived from the requirements document or may be documented in the design of development methods that support State Transition Notations (for example as supported by UML—Unified Modeling Language). This technique is also useful in Negative Testing whereby the tester can cover issues that either got overlooked or poorly specified in the specification document.

Consider the example below, which illustrates the principle behind State Transition Analysis



Static Testing

This does not involve executing or running the AUT but deals with inspection in isolation. We discussed this at the beginning of this chapter. Typical examples of Static Testing are: Code review and inspection, Code Walkthrough, Static Analyzers etc.

Thread Testing

The purpose here is to test the business functionality or business logic flow of the Application under Test (AUT) in an end-to-end manner, as if a user or an operator might interact with the system during its normal mode. Basically, in thread testing one tests the transactions processing as they are going to take place in the system/application designed.

As an example suppose we are doing thread testing of a student examination system, one particular thread could be processing the marks of a particular student. The thread testing process might involve the following steps:

- The operator logging onto the system
- Validation of the operator privileges
- Entry of the details of the student being processed
- Validation of the mark details
- Access the student database to verify the student
- Modification of the database to show that the student marks have now been processed
- Printing of the appropriate mark sheet to conclude the marks processing transaction

A point to be noted is that this technique contrasts with earlier testing phases (such as Unit and Integration testing) where the tester is likely to have been more interested in the correct functioning of the object under test rather than on its business logic

10.10.2 Nonfunctional Testing Techniques

First let us understand what do we mean by "nonfunctional requirements". Non-functional requirements are properties or qualities that the product must have (For example, *efficiency, integrity, reliability, usability, correctness, maintainability, expandability, flexibility, portability, re-usability* etc.) in order to enhance its "appeal". If not stated in the Software Requirements Specifications, these are generally the "nice to have" features of the software that the user is expecting.

Think of these properties or characteristics or "software quality factors" that make the product attractive, or usable or fast or reliable. For example, you may want your product to respond within a specified time, or display the calculated figures in certain colours and font. As another example, the product is expected to have a particular

appearance (i.e. the splash screen that comes upon starting the software, must use the standard company logo) etc. Thus, these properties of the product are not required because they are the functional requirements of the product (i.e. even without these, the software will work)—but they are there because the client wants the delivered software to operate in a certain manner. The vignette below further helps us understand “non-functional requirements”.

Non-Functional Requirements

“Look and Feel” requirements—*the intended appearance*

Based on intended user (level of comfort with software)—*Usability Requirements*

Performance Requirements—*how fast, big, accurate, safe, reliable etc.*

Social, Cultural and Political Requirements—*human factors, usability by handicapped people*

Legal Requirement—*conformance to applicable laws*

Security Requirement—*the security, confidentiality and integrity of the product*

Maintainability and Portability Requirements—*how changeable the product must be*

With this understanding, now let us have a brief look at what is involved in each of these non-functional testing.

Installation Testing

This is used to ensure that hardware and software have been correctly installed, all necessary files and connections, have been created, all appropriate data files have been loaded, system default settings have been correctly set, and interfaces to the other hardware systems/components or peripheral devices are all working fine. One also needs to ensure that the already installed software does not get affected due to additional installation of the hardware components.

Compatibility and Interoperability Testing

The purpose here is to verify that when the AUT runs in the live environment, its operation does not impact adversely on other systems, and vice versa. It is often required when a new system replaces an existing system that had previously interacted with one or more other systems. Interoperability Testing verifies that when the AUT runs in the live environment it is able to communicate successfully with other

specified systems (such as invoking other systems, passing data to other system and receiving data from other systems etc.)

Document and Help Testing

This is a frequently overlooked aspect. The testing of documentation of help facilities is often omitted from the testing plan; typically due to lack of time or resources. It is thought to be outside the scope of the testing process or through sheer carelessness. Contrary to the popular belief, these aspects of the AUT may be vital to its successful operation, and use of the system, particularly for new or naïve users. User documentation and help system information should be checked for conformance to the requirements specification document. Specific testing techniques involve documentation review, cross-referencing checks (e.g. against the document contents and index sections), and Thread Testing of typical user help scenarios.)

(Graphical) User Interface Testing

Sometimes, this may be clubbed with Usability Testing (discussed earlier). UI (User Interface) Testing evaluates how intuitive a system is from usage point of view. Issues pertaining to navigation, usability, commands and accessibility are considered. User interface functionality testing examines how well as UI operates to given specifications. Main areas covered in UI Testing are:

- Look and feel
- Navigation controls/navigation bar
- Navigation Branching
- Instructional and technical information style
- Images
- Tables and Forms

Fault Recovery Testing

Fault Recovery Testing verifies that following an error or exception (such as a system crash caused by loss of power) the AUT can be restored to a normal state, such as the initial state of the application when it is first executed, and that the AUT can continue to perform successfully. Fault Recovery Testing may also be used to verify the successful rollback and recovery of the data used or manipulated by the AUT following an error or exception.

Performance Testing

Performance Testing is a problem area because system performance is often poorly specified. Systems performance can be measured using a variety of different criteria, including:

- System response time
- External interface response time
- Central processor unit (CPU) utilization
- Memory utilization

The primary goal of Performance Testing is to develop effective enhancement strategies of maintaining acceptable system performance. Performance testing is an information gathering and analyzing process in which measurement data are collected to predict when load levels will exhaust system resources.

In conducting Performance Testing it is essential to have a suitable performance model that specifies what aspect of the system is being tested, what the performance requirements are, how they can vary under typical system usage, and how they will be tested.

Another challenge in Performance Testing is the difficulty in accurately representing the test environment. For example, if there are likely to be a large numbers of concurrent users and/or large amounts of data, it may be impractical or infeasible to set up and conduct realistic performance testing. Because of this and similar other difficulties, testing tool support is essential for Performance Testing, Stress Testing and Load (Volume) Testing. It should be noted that the terms *performance testing*, *stress testing* and *load/volume testing* are related.

Reliability Testing

This involves tests to ensure the robustness and reliability of the AUT (Application Under Test) under typical usage. It typically includes integrity tests and structured tests. Integrity tests focus on verifying the AUT's failure frequency and compliance to language, syntax, and resource usage. For example, a particular unit could be executed repeatedly using a test harness (this term was introduced in section 10.7) to verify that there are no memory leak problems. These tests can be designed, implemented, and executed during any testing phase. Some related IEEE definitions are provided below in reference to *reliability testing* and *performance testing* (previous section).

Mean time between failure (MTBF) The expected or observed time between consecutive failure in a system or component

Up Time The period of time during which a system or component is operational and in serviced; that is, the sum of *busy time* and *idle time*

Busy Time In computer performance engineering, the period of time during which a system or component is operational, in service, and is use.

Idle Time The period of time during which a system or component is operational and in service but not in use.

Mean time to repair (MTTR)

The expected or observed time required to repair a system or component and return it to normal operation

Down Time The period of time during which a system or component is not operational or has been taken out of service

Another important point to note is "Quality and Reliability". Dictionary meaning of quality is "a degree of excellence" or "superiority of mind". If a software product is of high quality, it will meet the customer's needs ("Fitness for use" viewpoint of quality). The customer will feel that the product is excellent and superior to his other choices. Here is where the hitch comes! Software testers often fall into the trap of believing that *quality and reliability* are the same thing. They feel that if they can test a program until it is stable, dependable, and reliable, they are assuring a high quality product. Unfortunately, that is not necessarily true. Reliability is just one aspect of quality.

The User's mind

A software user's idea of quality may include much more than just reliability. He is thinking of the breadth of features, the ability of the product to run on his/her older version of hardware, the software supplier's post sale support, may be even the colour of the packaging! Reliability, or how often the product crashes or trashes his data, may be important but not always

Thus, to ensure that a software program is of high quality and reliable, a tester must both verify and validate throughout the product development process.

Security Testing

Security issues are of the highest concern to many organizations especially those with Web-based products and services. Despite this fact, security testing is often the least understood, and least well-defined test type. Security testing efforts require domains of expertise beyond traditional software testing. Depending on the intended role of the AUT, there may be requirements that specify the need to ensure the confidentiality, availability and integrity of the data and software. Security testing is intended to test whether the features implemented within the AUT provide this required level of protection. Security testing is mainly

concerned with establishing the degree of traceability from the requirements through to implementation, and in the validation of those requirements. For rigorous security testing, dedicated test teams with expertise in the security areas are required. Security tests can be designed, implemented, and executed within any testing phase but are typically conducted at System Test and re-run during User Acceptance Testing.

Next, we discuss *stress testing*, *load testing* and *volume testing*. Often the three terms get mixed up. It should be noted that these terms are related but not the same.

Stress Testing

The purpose of this is to examine the ability of the system to perform correctly under instantaneous peak loads with the aim of identifying defects that appear only under such adverse conditions. For example, if the specification for the AUT states that the system should accept 40 concurrent users, what happens if a 42nd user attempts to log on? Similarly, what when 40 users attempt to log on simultaneously? Thus, Stress Testing evaluates the behaviour of system that is pushed beyond their specified operational limits (this may well be above the stated requirements). It evaluates responses to bursts of peak activity that exceed system limitations. As said above, determining whether a system crashes or recovers gracefully from such conditions is a primary goal of Stress Testing. Simulation is frequently used method in Stress Testing. It is also worth considering commercial tool support for Stress Testing.

Load Testing/Volume Testing

Load Testing (also known as Volume Testing) evaluates system performance with pre-defined load level. Load Testing measures how long it takes a system to perform various program tasks and functions under normal, or pre-defined conditions. Bugs are reported when tasks cannot be executed within the time limits (as defined by the product management or marketing group). Because the objective of Load Testing is to determine if a system performance satisfies its load requirements, it is pertinent that minimum configuration and maximum activity levels be determined before testing begins.

As said at the start of this section, the three terms: Performance Testing, Load (Volume) Testing and Stress Testing are often used interchangeably; each of them addresses a different objective as explained on the next page.

Stress testing is running the software under "less-than-ideal conditions" – low memory, low disk space, slow CPUs, slow modems and so on. The goal is to starve the software for the resources it is looking for. **Load Testing** is the opposite of stress testing. With Load testing, instead of starving the software, you feed it all that it can handle – operate the software with the largest possible data files. If the software system operates on peripherals such as printers or communication ports, connect as many as you can. If you are testing an Internet server that can handle thousands of simultaneous connections, do it. The idea is to max out software system's capabilities by loading it down. To this extent, sometime people like to use the terms Performance Testing and Load (Volume) Testing interchangeably. A load test is done to determine if the system performance is acceptable at the "pre-defined" load level; a Performance Test is done to determine the system performance at various load levels.

Usability Testing

Specific techniques employed in Usability Testing include:

- Conformance checks—testing the application against agreed user interface standards
- User-based surveys—psychometric testing techniques to analyze user perceptions of the system
- Usability Testing—where users are asked to perform a series of specified business tasks with the AUT under controlled conditions in order to test the usability goals or requirements of the AUT

10.11 Test Metrics

Having discussed various testing techniques, now let us turn our focus to test measures and metrics. (The topic of software measurement and metrics has been discussed in detail in Chapter 5. The meaning of terms *measures* and *metrics* has been explained there.) Test measures and metrics are important for improving the testing process. Basically the measures and metrics for testing can be divided into the following broad categories:

[A]

- Extent of testing
- Testing effectiveness
- Assessment of testing
- Resources consumed in testing

[B]

- Testability measures
- Test control metrics
- Test Result Metrics

The table below explains category [A]. The list below is only an illustrative list and not an exhaustive one. There are many more measures and metrics that can be constructed. For a detailed discussion of terms *path coverage* and *statement coverage* etc., readers may like to refer [10] in the reference list at the end of this chapter.

Table 6: Test Metrics Category 1

Measure/Metric	What it shows	Description
Statement Coverage (Instructions Exercised)	Extent of Testing	Number of instructions in the program that were executed during the test process
Number of tests		Number of tests required to evaluate a module
Path Coverage (Number of paths tested)		Number of logical paths that were executed during the test process
Detected production errors (errors found after the software has been moved in the production environment)	Testing Effectiveness	Number of errors detected in the application when deployed in the end use environment
Defects uncovered in testing		—
Business value of testing		Money saved by uncovering defects early
Re-run analysis	Resources consumed in testing	Re-run hours associated with undetected defects (system down time)
Cost of testing		Ratio of cost of testing to the total system cost. Amount of resources (humans, equipment etc.) deployed for testing activity and their notional cost
Average cost of locating a defect		Cost of testing versus the number of defects located in testing
The extent of adherence to testing budget	Assessment of testing	Planned cost of testing v/s actual cost of testing
Loss value of test		Effect of testing in reducing losses as related to the resources processed by the system
Scale of ten		People's assessment of effectiveness of testing on a 10-point scale (1 = POOR, 10 = OUTSTANDING)
Requirement coverage		Monitoring and reporting on number of requirements exercised and tested

10.11.1 Testability measures

In practice, commercial projects are always under tremendous pressure of delivery dates. Although life cycle testing (testing throughout the development life cycle) is a good concept, most of the time test teams have to struggle to draw management attention to provide them adequate testing time and testing resources. Being able to predict the testability of the software and estimating the testing effort early in a project comes in handy in such circumstances. Testability metrics can play a significant role in this perspective. The discussion that follows is from this point of view.

Testability metrics measure features of a test item, which are likely to make the item easy (or difficult) to test. Testability metrics are useful to assess the ease of all testing phases (unit testing, integration testing, system testing). Useful testability metrics are those that can be derived from requirements and design documents. This way one can use them to assess future testability during early stages of systems development.

Testability metrics for integration testing may be based on module linkage metrics such as *fan-in* (number of modules calling a module), *fan-out* (number of modules called by a module), and counts of reads and writes to common data items. *Testability metrics for code* include metrics based on module control flow such as McCabe's *cyclomatic complexity number* (refer Chapter 5. For a detailed discussion on the topic of "Cyclomatic Complexity" reader can have a look at [15] quoted in the reference list at the end of this chapter). *Cyclomatic Complexity* identifies the number of independent control flow paths.

In summary, testability metrics are usually size or structure measures used for the purpose of assessing ease of testing. They are used:

- To identify potentially un-testable (or difficult to test) components at an early stage in the development process. In this context, poor testability is equated to a large number of control flow paths. This provides the opportunity to redesign components and/or provide additional testing effort.
- To assess the feasibility of test specifications in terms of the provision of adequate test cases and appropriate test completion criteria.
- To estimate the effort and schedule of test activities. This assumes that predictive models are available to relate testability metrics to testing effort and duration.

10.11.2 Test control metrics

These are of three main types:

1. Test Resource Metrics (see Table 6 in the previous section)
2. Test Coverage Metrics (see Table 6 in the previous section)
3. Test Plan Metrics